# Software Ecosystems

## Analysis of Godot Game Engine
## & Designing a Learning Platform

Course Code: KSSOECO1KU

Course Name: Software Ecosystems - combining user centered design, organisational and technical dimensions

Course Manager: Yvonne Dittrich

**Writers:**
Aditya Fadhillah, Amal Hussein, Helga Ingrun Maria Thurn, Parnuna A D Rasmine R Brønlund

# Table of Contents

**Introduction** ..... **3**

**Part 1: Analysis of Godot as Software Ecosystem** ..... **3**

*1.1 Godot as Software Ecosystem* ..... *3*

*1.2 Godot's Modular Architecture through SRMA Perspective* ..... *4*

*1.3 Stakeholders in the Godot Engine Ecosystem* ..... *10*

*1.4 Business Model Analysis* ..... *13*

*1.5 UX Evolution in Godot's Software Ecosystem: From Tool to Co-Design Platform* ..... *17*

**Part 2: Designing a Learning Platform** ..... **21**

*2.1 Problem Statement* ..... *21*

*2.2 Theoretical Background* ..... *21*

*2.3 Stakeholders* ..... *22*

*2.4 Design Research Methodology* ..... *23*

*2.5 Business model* ..... *27*

*2.6 Current Godot Website Architecture and Proposed Addition* ..... *27*

*2.7 Prototype and Design Choices* ..... *29*

**Reflection** ..... **32**

**References** ..... **33**

**Appendix** ..... **37**

*1. Ecosystem* ..... *37*

*2. Personas* ..... *38*

*3. User Story* ..... *39*

*4. Surveys:* ..... *39*

# Introduction

As part of the Software Ecosystem (SECO) course we had to analyse an open-source software ecosystem and suggest in what way we could improve it. After brainstorming different ideas and systems, we decided on Godot a game engine, that none of us had previous experience with. This gave us the opportunity to understand a completely new field and apply the freshly acquired knowledge from the course.

Godot is a modern open-source software ecosystem where technical architecture, community practices, and governance all shape how the platform grows and evolves. The engine is developed and maintained by a global network of contributors ranging from the core team to plugin creators, educators, studios, hobbyists, and donators. Together they form a diverse ecosystem that depends on, transparent decision-making and continuous knowledge exchange.

The report is divided into two main parts. Part 1 presents our analysis of Godot as a software ecosystem, including its architecture, flexibility, stakeholder landscape, business model, and UX development. Part 2 introduces the design feature, explaining the problem we address, the theoretical background, the design methodology, and the prototype we created. Together, these parts show how a deeper understanding of the ecosystem can support practical design decisions that strengthen learning, participation, and long-term sustainability.

# Part 1: Analysis of Godot as Software Ecosystem

## 1.1 Godot as Software Ecosystem

Godot is an open-source software ecosystem that provides a complete platform for game development. Godot is released under the Massachusetts Institute of Technology (MIT) licence, which guarantees free use, modification and redistribution of the engine's source code. As a vertical platform, Godot offers all necessary tools to create games from conception to completion within one integrated environment, distinguishing itself from horizontal platforms that address only specific development functions. The engine's development follows a community-driven model where numerous small contributions from a global community of volunteer developers merge into a substantial platform (Docs-Godot Engine, n.d.-a). Figure 1 summarises this ecosystem by showing how governance and funding (the Godot Foundation), core development, community contributors, external tooling and distribution platforms interact around the engine and its users. A more detailed version is available in the "Ecosystem Diagram Design" figure on Figma (click the link in Appendix to see "Final figures and models").

Godot primarily targets indie developers, small studios, and hobbyists, offering a compelling free alternative to Unity with complete source code access. The platform includes an editor, scripting environment, asset management and deployment pipeline, and runs on Windows, macOS and Linux with export options to web, mobile and consoles. Multi-language support includes GDScript (a language designed for Godot), C# and C++. Through the Asset Library,

community contributors can publish tools and templates that other users can integrate into their own projects, reflecting the collaboration shown in Figure 1.

The engine's strengths include no licensing fees, full source code access, active community engagement, and transparent development. However, challenges include a smaller market share compared to commercial alternatives and fewer features optimized for AAA game development. This approach puts community innovation ahead of commercial interests, making Godot fundamentally different from proprietary engines.

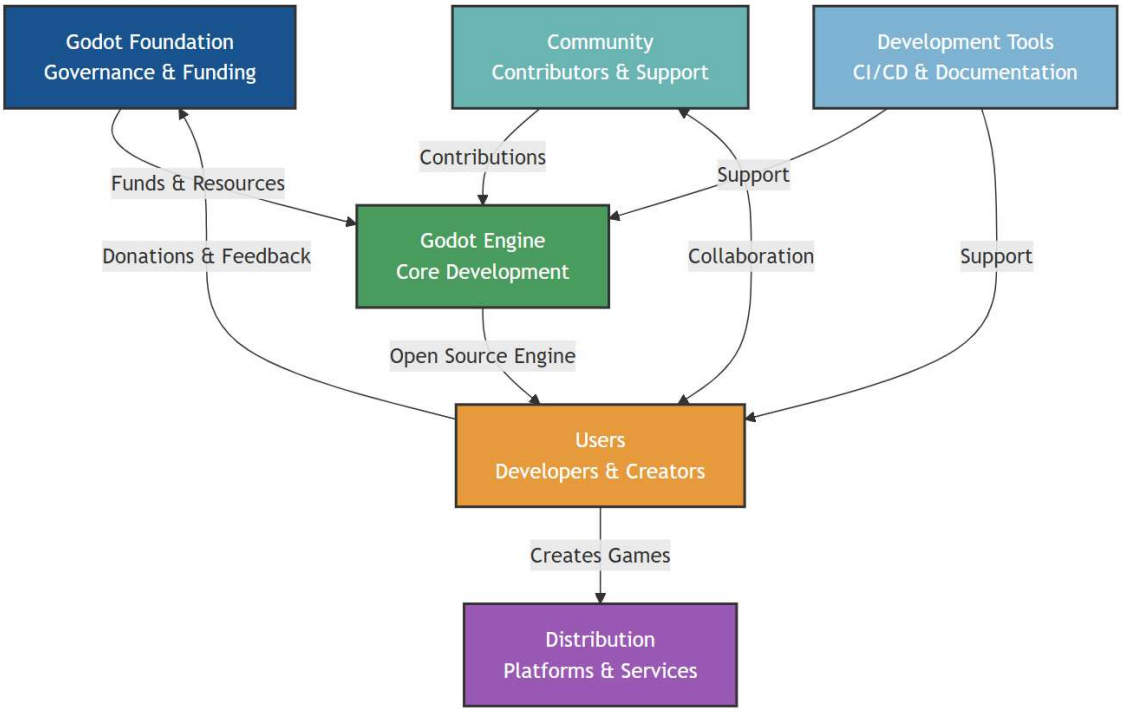



*Figure 1 Overview of the Godot software ecosystem, illustrating the relationships between governance structures, community actors, technical architecture, and external stakeholders.*

## 1.2 Godot's Modular Architecture through SRMA Perspective

Godot's architectural design demonstrates characteristics aligned with Pruijt & Brinkkemper's (2014) definition of Semantically Rich Modular Architectures (SRMA). They describe these as modular architectures containing *"modules of semantically different types, like layers and components, constrained by rules of different types"* (p. 2). SRMA theory gives us language for understanding how Godot's components interact and why certain architectural choices matter for long-term maintenance.

Godot's architecture combines a node-based composition system, server-based processing, scene management hierarchies, and a signal communication layer. These components work together to create what Pruijt & Brinkkemper, (2014, p. 2) call *"modules with specific semantics (that) enhance the expressiveness of a modular architecture and support architecture reasoning"*. Figure 2 helps make this visible by showing Godot organised in layers (Core - Scene - Servers - Drivers), where higher-level game objects and editor-facing elements sit above specialised low-level processing systems. Figure 4 presents the same idea in a simplified way: the editor and the scene/node system on top, a server layer underneath

for performance-critical work, and an engine foundation and extension systems that support both.

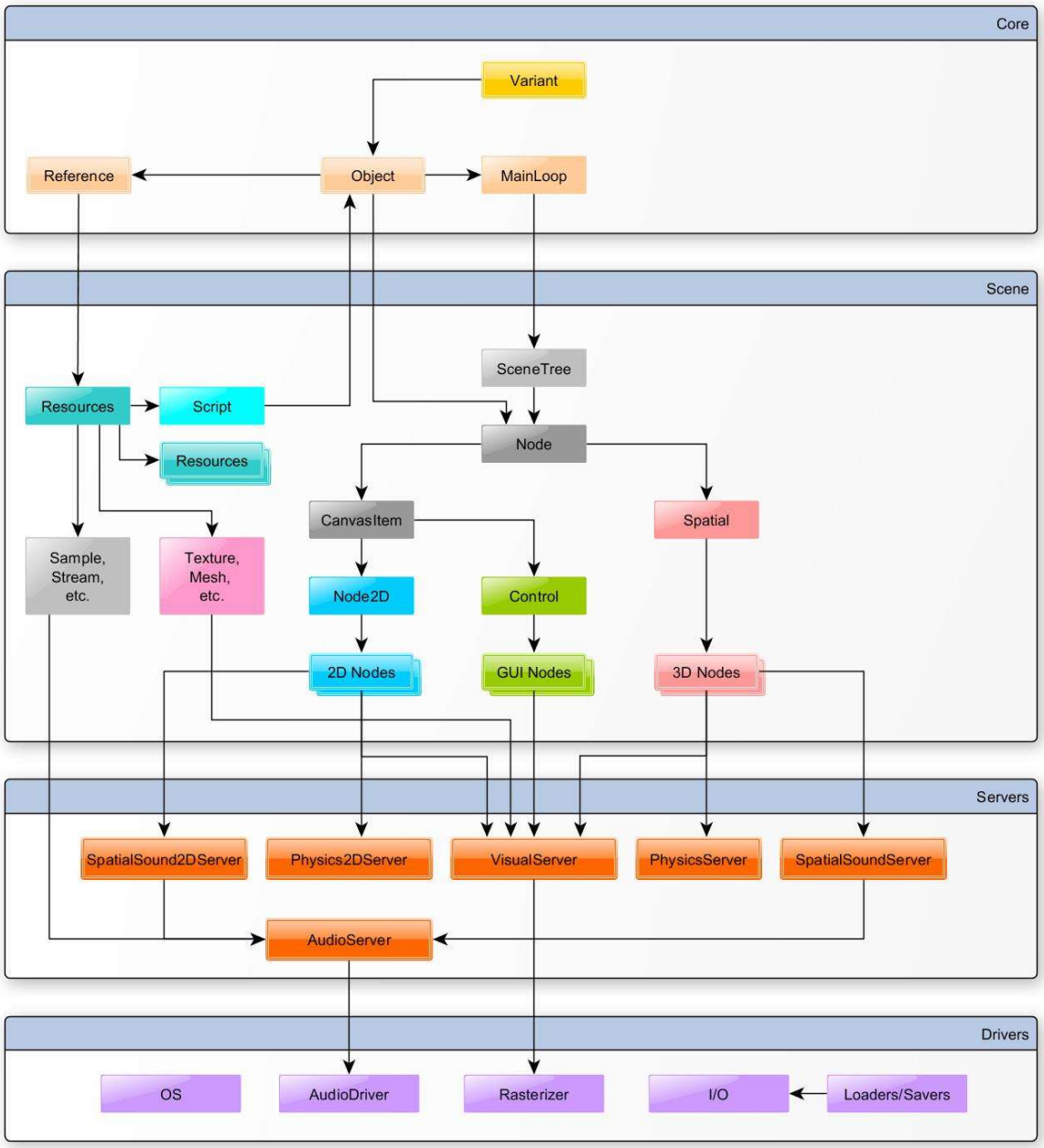
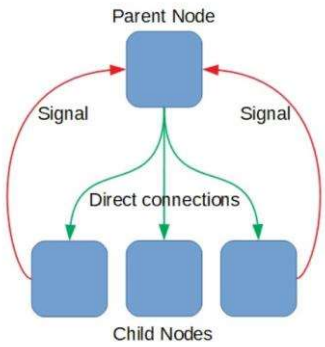



*Figure 2 The diagram describes the architecture used by Godot (Godot engine, n.d, -x)*

**Node and Scene System.** As shown in Figure 2 and simplified in Figure 4, most day-to-day development happens in the "Scene" layer, centred on the SceneTree and the Node hierarchy. Godot uses a node-based architecture where every element in a project is a node arranged in a tree structure. Nodes act as modular building blocks with specific responsibilities (for example, a sprite node draws an image, a button node handles clicks, and an audio node plays sounds). Each node type inherits from a base Node class and provides clear interfaces, showing what Pruijt and Brinkkemper (2014, p. 3) call the facade convention where components have well-defined entry points. The scene system groups related nodes into reusable compositions, which is why Figure 4 places "Scene & Node System" at the centre of the architecture. A Player scene might contain a CharacterBody2D for physics, a Sprite2D for

visuals, a CollisionShape2D for hit detection, and an AnimationPlayer for animations. Scenes can be composed of other scenes, creating hierarchical patterns (Docs-Godot Engine, n.d.-b). Pruijt and Brinkkemper (2014, p. 2) would call this *"logical clusters"* with *"clearly assigned responsibilities."* The tree structure makes relationships explicit: parent nodes coordinate their children (for example, moving a parent moves everything under it), which helps break complex behaviour into manageable parts. Importantly, Figure 4 also highlights that the Godot Editor itself is built using the same node/scene model, showing that this composition approach scales from simple games to complex tooling.




**Signal System.** Figure 3 visualises the two ways nodes interact: direct connections (tighter coupling) versus signals (looser coupling), following the "is-not-allowed-to-use" rules fundamental to SRMA theory (Pruijt & Brinkkemper, 2014, p. 3). The green arrows in the figure represent direct calls from a parent to its child nodes, where components depend on each other's concrete interfaces. The red arrows represent signals flowing back as events: a node emits a message when something happens, and any other node can react without being directly referenced.

*Figure 3 Node signal (GDScript, 2025)*

For example, a Button node can emit a "pressed" signal without knowing which nodes will respond, and any node can connect to this signal without the Button needing awareness of its subscribers (Docs-Godot Engine, n.d.-c). This observer pattern promotes modularity by letting nodes stay independent while still communicating. A health system might emit a "health_changed" signal that both the UI and audio system listen to, without these systems knowing each other. Adding new systems like analytics tracking then requires no changes to the health system itself.

**Server Architecture.** Figures 2 and 4 make the node–server split explicit: nodes and scenes sit in the "Scene" layer; while rendering, physics and audio are executed in the "Servers" layer underneath. Alongside the node system, Godot uses specialised servers (RenderingServer, PhysicsServer2D/3D, AudioServer) that handle low-level processing through command-based APIs. In Figure 4 this interaction is labelled "Commands/Data Exchange," capturing how high-level nodes issue requests while servers perform the heavy work, potentially in parallel threads. This separation creates two distinct layers: an accessible interface layer and a performance layer. Developers who need direct control for optimisation can access the server APIs, but most development remains at the node level (Docs-Godot Engine, n.d.-d). Pruijt and Brinkkemper (2014, p. 2) would describe this boundary between the node interface and server implementation as semantic differentiation between module types.

**Extension System.** Figure 4 highlights extension as a distinct architectural layer, showing "Extension Systems" (GDScript, C#/.NET, and GDExtension C++) alongside the plugin

system for editor extensions. This illustrates that Godot's modularity is not only inside the engine, but also in how new functionality is added around it. GDScript provides built-in scripting tightly integrated with the node and scene workflow, using Python-like syntax for accessibility. C# support works through the .NET runtime, giving developers strong typing and access to the broader .NET ecosystem. GDExtension allows native C++ code to extend the engine without recompiling it, which is particularly relevant for performance-critical modules that still need to plug into the same node-based interface. Figure 4 also separates "Plugin System (Editor Extensions)" to show that users can extend not just gameplay features, but the editor itself by adding custom tools, importers and node types that behave like built-in components.

Experienced users need deeper control for performance or integration. They extend Godot through plugins and GDExtension modules (Docs-Godot Engine, n.d.-e). These extensions provide clear, stable interfaces while hiding their internal implementation. Most developers then use them like any other node, resource or editor tool, without dealing with server APIs.

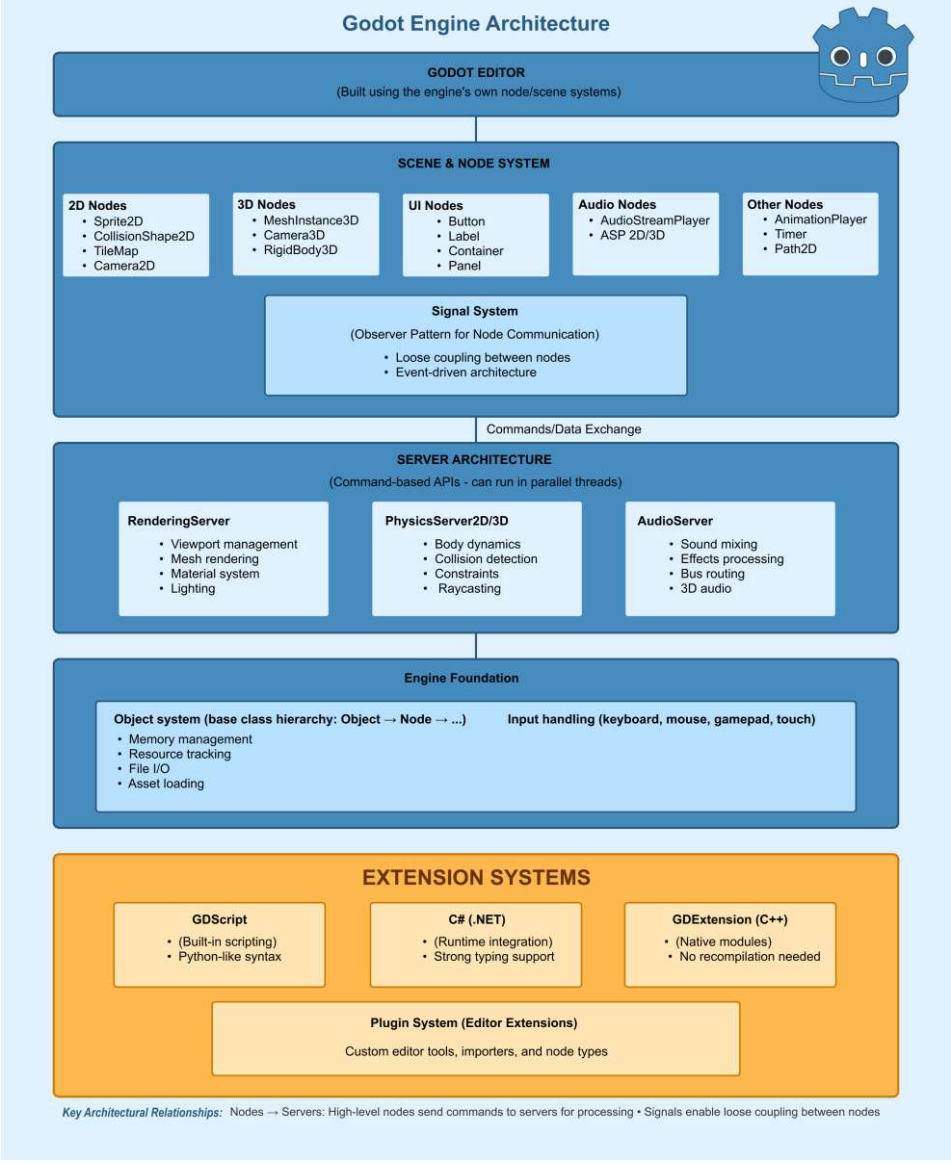



*Figure 4 Godot Engine's modular architecture. Based on Godot documentation and Pruijt & Brinkkemper (2014).*

### 1.2.1 Architectural Tensions and Flexibility

Applying SRMA theory to Godot reveals tensions between accessibility and architectural strictness. Unlike enterprise software architectures that enforce rigid module boundaries, Godot's design philosophy focuses on rapid iteration and developer freedom.

The node system provides multiple mechanisms for interaction; this flexibility means Godot suggests architectural constraints rather than enforcing them. Developers can choose loose coupling through signals or tight coupling through direct references, depending on their needs. In everyday work this supports flexibility: a typical Player scene can be extended with new child nodes, resources or signal connections (for example, a dodge mechanic or health UI) without rewriting existing logic, and higher-level wrappers can shield most of the project from internal changes.

Pruijt and Brinkkemper (2014, p. 3) note that explicit architectural rules help "*reduce the number of rules that need to be specified.*" Godot does the opposite: it provides many ways to build interactions, requiring developers to set their own standards. Patterns and best practices (such as “call down, signal up” for node communication) are encouraged but not enforced, so teams must decide how strictly to separate concerns and where to place abstraction layers that localise future changes (Docs-Godot Engine, n.d.-f)

These are architectural tensions rather than problems. The flexibility helps game developers experiment and iterate quickly. However, developers must maintain their own architectural discipline instead of relying on technical restrictions.

### 1.2.2 Evolutionary Challenges

The migration from Godot 3.x to 4.x shows challenges in long-term software evolution. The rendering architecture underwent a complete rewrite, resulting in breaking changes that affected the entire ecosystem (Docs-Godot Engine, n.d.-g). Node types changed names (KinematicBody became CharacterBody2D), APIs changed, and projects needed migration work.

Breaking changes create dependencies between groups. Plugin developers wait for the core team to release updates, game developers wait for updated plugins, and players wait for migrated games. Sestoft and Vaucouleur (2008) map these upgrade dependencies in software product ecosystems, see Figure 5.

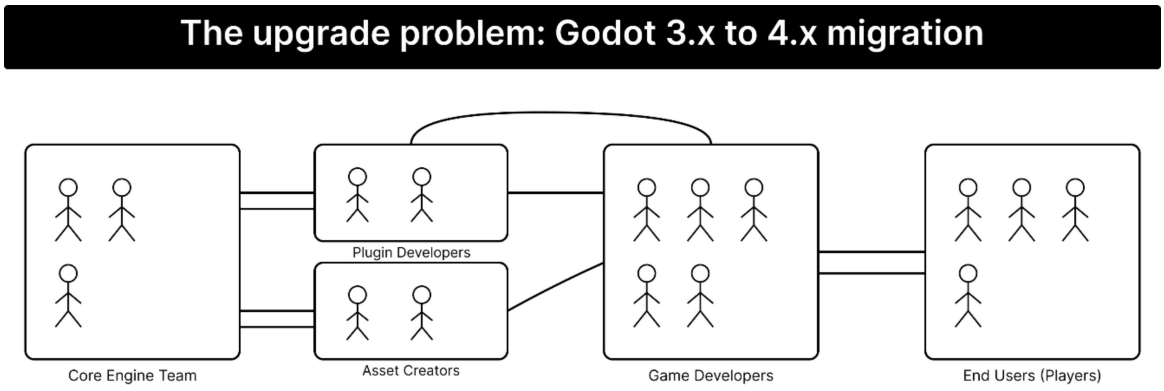



*Figure 5 Upgrade dependencies in Godot's ecosystem, adapted from Sestoft & Vaucouleur (2008).*

As Unphon & Dittrich (2010, p. 2220) observed, *"the main issue when developing software products is not to implement the design architecture... but to maintain the architecture in a viable state when evolving the software."* The Godot 4 migration shows this tension: architectural improvements for the future required disrupting the present ecosystem.

Documentation shows Godot's distributed development model. User-facing documentation explains how to use nodes, scenes, and signals extensively. However, architectural documentation describing internal structure and design rationale exists primarily in source code and developer discussions. This matches with Unphon and Dittrich's (2010, p. 2218) observation that *"official architecture documentation quickly becomes outdated, while the actual architecture exists primarily in source code and community discussions."*

Rather than a single *"walking architecture"* figure as Unphon and Dittrich describe in commercial software development, Godot spreads architectural knowledge across core maintainers, user documentation, community forums, and GitHub discussions. This creates opportunities like shared expertise and community feedback but also challenges around coordination and keeping the architecture consistent as it evolves.

### 1.2.3 Layered Complexity in Godot's Architecture

Godot's architecture separates into complexity layers, moving from accessible visual building blocks at the top to low-level server operations. Most projects gain flexibility at the higher layers by combining and configuring these building blocks, rather than by modifying the engine itself.

At the most accessible layer, users work with the visual editor and ready-made node types, such as sprites, buttons or audio players, which can be combined through drag-and-drop. The signal system lets these elements react to each other (for example, a character taking damage when a hit is detected) without requiring complex code (Docs-Godot Engine, n.d.-b). A typical gameplay element can be extended by adding new components and connecting them to existing signals, instead of rewriting the entire setup.

Moving beyond tutorials requires deciding where logic should be placed and how to define methods and signals. When architectural boundaries are treated as stable contracts through scripting, they double as safe customisation points (Docs-Godot Engine, n.d.-f): designers can rearrange collision and decoration layers or plug in systems such as achievements or save system that simply listen to existing signals.

More advanced work interacts with lower layers such as engine APIs or native extensions (Docs-Godot Engine, n.d.-d; n.d.-e) where complexity is hidden behind small helper functions or modules, so that most of the codebase still operates at the level of nodes and signals.

Overall, Godot’s architecture is experienced differently depending on whether someone is building the engine, extending it, or learning it through the docs and tutorials. Because of that, how the architecture evolves is tightly linked to the people and organisations around it. The next section the technical structure to the key stakeholders and their roles are discussed.

## 1.3 Stakeholders in the Godot Engine Ecosystem

In organizational strategy, stakeholders broadly include any group or individual who can affect or is affected by an organization's objectives (Bryson, 2004). Effective stakeholder analysis helps ensure that key interests are recognized and managed. (Bryson, 2004) outlines methods such as the power - interest grid - mapping stakeholders by their influence and interest - to prioritize attention. He also describes a basic stakeholder analysis technique: identify each stakeholder's expectations and assess how well the project meets them, highlighting "quick wins" to satisfy key players and long-term issues to address. By understanding who the stakeholders are and what they need, managers can build more robust strategies and coalitions.

In the context of software ecosystems - networks of actors around a shared technological platform - stakeholder roles often follow certain patterns. Manikas & Hansen (2013) identify five major actor categories: orchestrator, niche players, external actors, vendors, and customers. The orchestrator (also termed keystone, hub, or platform owner) manages and governs the ecosystem, providing the core platform and setting rules, roadmaps, and technical processes. Niche players complement and extend the platform's functionality by developing add-ons or components. External actors are those external to the platform's governance who still build on or contribute indirectly. Vendors build and sell products or services based on the platform. Finally, customers (end-users) are those who obtain and use the products of the ecosystem.

Alves et al., (2017) note that successful ecosystem governance must enable value creation for all players, coordinate their interactions, and strike a balance between openness and control. The orchestrator should allow enough autonomy to attract niche contributors, while maintaining enough control to ensure coherence - a balance that produces a healthy, innovative community.

| Category (M&H 2013) | Stakeholder Examples (Godot) | How does stakeholder judge performance? | How well is org. doing? | Quick actions to satisfy stakeholder | Longer-term issues with stakeholders | How does stakeholders influence organization | What does organization need from stakeholder? | Ranking (power, legitimacy, attention) |
|---|---|---|---|---|---|---|---|---|
| **Orchestrator** | Godot Core Team; Godot Foundation | Stability of governance, technical progress, funding transparency | Good | Communicate roadmap clearly, publish funding reports | Risk of burnout, governance conflicts | Sets direction, allocates funds | Commitment and stewardship | High (power, legitimacy) |
| **Niche Players** | Community plugin & tool developers; Asset library contributors | Ease of extending engine, visibility of contributions, recognition | Fair | Improve documentation, highlight contributors | Maintaining motivation, avoiding fragmentation | Add value through ecosystem, fill feature gaps | Continuous contributions, innovation | Medium (attention, legitimacy) |

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| **External Actors** | Community testers & bug reporters; Donors; Educational partners | Responsiveness to bug reports, gratitude for donations, usefulness in teaching | Fair | Acknowledge contributions, faster bug triage | Sustaining donations, institutional support | Provide indirect resources, advocacy | Donations, feedback, educational adoption | Low–Medium (attention, legitimacy) |
| **Vendors** | Game studios using Godot; W4 Games; Asset store sellers | Engine reliability, performance, commercial viability | Fair-Good | Offer stable LTS versions, pro support | Monetization conflicts, dependency on ecosystem | Generate commercial credibility, bring funding | Use cases, validation, commercial adoption | High (power, attention) |
| **Customers (End-Users)** | Game developers adopting Godot; Players of Godot-made games | Developer: usability, docs, features. Player: quality of games produced | Fair | Better tutorials, docs examples, UX improvements | Keeping up with industry standards | Drive adoption rates, reputation | Adoption and community growth | High (attention, legitimacy) |

*Figure 6 Basic Stakeholder Analysis in combinations with SECO actors' classifications by (Manikas & Hansen, 2013, p. 1301)*

The Godot ecosystem is shaped by a diverse set of stakeholders whose roles and practices collectively sustain and evolve the platform. The basic analysis in Figure 6 highlights how different actor groups, categorized by Manikas & Hansen (2013), Orchestrators, Niche Players, External Actors, Vendors, and Customers - contribute distinct tasks, judge organizational performance, and exert influence.

Stakeholders influence the SECO through their ability to steer direction, extend functionality, provide resources, and validate adoption. The Orchestrators (Godot Core Team and Foundation) set the technical roadmap, manage funding, and govern processes - exerting high power and legitimacy (Governance-Godot Engine, n.d.). Niche Players (plugin and tool developers) expand the ecosystem by creating add-ons, thereby addressing specialized user needs and filling functional gaps (Contributing-Godot Engine, n.d.). External Actors (donors, testers, educators) contribute indirectly by offering feedback, resources, and legitimacy in educational contexts. Vendors (studios, W4 Games) validate Godot's commercial viability and generate professional credibility (W4Games; Godot Foundation, n. d.). Finally, Customers, in this case, developers and players contribute to Godot's adoption and reputation through widespread use and visible success in the market, as evidenced by the growing number of Godot-based games on Steam and their strong reception in game jams and commercial releases (Burton, 2025).

The actors' practices differ across categories. According to Dittrich (2014), software ecosystems thrive on distributed, practice-based design work, where communities engage in mutual learning and continuous negotiation. In Godot, Orchestrators emphasize governance and integration practices, ensuring coherence of contributions. Niche Players rely on

collaborative design practices, where experimentation and iterative release of plugins extend the ecosystem's capabilities. Vendors adopt professionalized development methods, emphasizing stability and support for commercial applications. Customers engage in end-user development practices, providing experiential feedback that shapes usability priorities.

Figure 7 applies the power versus interest grid (Eden & Ackermann, 1998) to classify the stakeholders in the Godot software ecosystem (SECO). This visualization highlights how different actors are positioned based on their ability to influence the ecosystem (power) and their degree of commitment to it (interest). The Players quadrant (high power, high interest) includes the Orchestrators and Vendors such as W4 Games and professional studios.

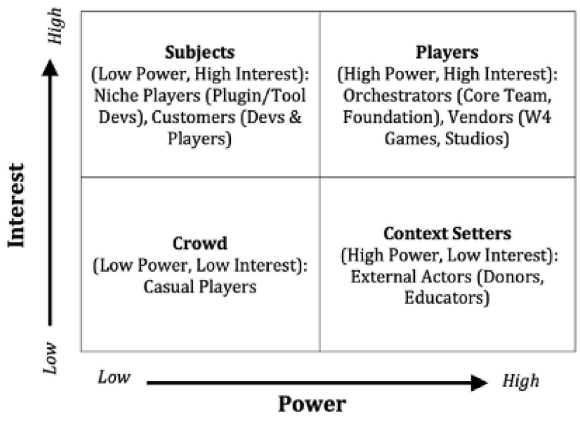



*Figure 7 Power versus interest grid (Eden and Ackermann, 1998)*

These stakeholders combine formal authority and commercial validation with a strong commitment to Godot's success, making them important for strategic direction and sustainability. Their influence requires careful management and collaboration, as conflicts or withdrawal could significantly destabilize the SECO.

The Subjects quadrant (low power, high interest) is populated by Niche Players and Customers. Although these actors have limited formal decision-making power, their high level of interest drives innovation and adoption. Their contributions and feedback are essential for ecosystem vitality, but their dependence on orchestrators for recognition and integration creates potential tensions, consistent with Dittrich's (2014) observation of practice-based negotiation in ecosystems.

Context Setters (high power, low interest), such as donors and educational partners, exert influence by legitimizing Godot through financial or institutional support, yet their day-to-day involvement remains peripheral. Their role is strategic rather than operational, which makes them critical in moments of resource scarcity or external validation.

Finally, the Crowd quadrant (low power, low interest) consists of casual players who consume Godot-made games without actively contributing to the engine's development. While their individual impact is limited, collectively they shape the broader reputation of the ecosystem and can signal potential growth in user adoption.

Overall, Figure 7 illustrates the heterogeneous stakeholder landscape of Godot. It shows that sustaining the ecosystem requires balancing the governance authority of Players, the creative energy of Subjects, the external legitimacy of Context Setters, and the latent potential of the Crowd. This aligns with prior research emphasizing that software ecosystems are not only technical but also socio-political structures shaped by diverse actor roles and dependencies (Jansen et al., 2013).

While the power-interest grid offers a useful classification, it also has limitations when applied to software ecosystems. First, roles are not always static: for example, a highly engaged plugin developer may shift from Subject to Player if their tools gain widespread adoption, altering their relative power. Second, interests and power can overlap across categories - vendors may simultaneously act as donors (Context Setters) or maintain plugins (Niche Players). This fluidity challenges the clear-cut quadrant distinctions suggested by the model. Finally, the grid emphasizes structural positioning but pays less attention to the social and cultural practices and the network dependencies that sustain ecosystems. Thus, while Figure 7 clarifies stakeholder distribution, it should be complemented with qualitative insights into how these actors interact, negotiate, and co-create within Godot's socio-technical landscape.

Overall, the stakeholder analysis shows who is involved in the Godot ecosystem and how they influence it. What it does not yet explain is how value flows between these actors. In the next section, we therefore shift to a business model perspective and use the Business Model Canvas and the Multisided Value Proposition Canvas to describe how Godot, as a platform, creates, delivers, and captures value across the different stakeholder groups.

## 1.4 Business Model Analysis

According to Osterwalder & Pigneur (2010), a business model is defined as *"the rationale of how an organisation creates, delivers, and captures value"* (p. 14). The Business Model Canvas (BMC) serves as a visual and strategic tool that breaks down this logic into nine key building blocks, covering customers, value propositions, infrastructure, and finance. The purpose of the canvas is to create a shared language that enables the analysis and development of business models in a simple and systematic manner.

In the following, we use this definition as a starting point and apply the Business Model Canvas with its nine building blocks to analyse the Godot software ecosystem (see Figure 8). Finally, we contextualise the analysis using the Multisided Value Proposition Canvas to compare value creation across the platform's different stakeholders (see Figure 9).

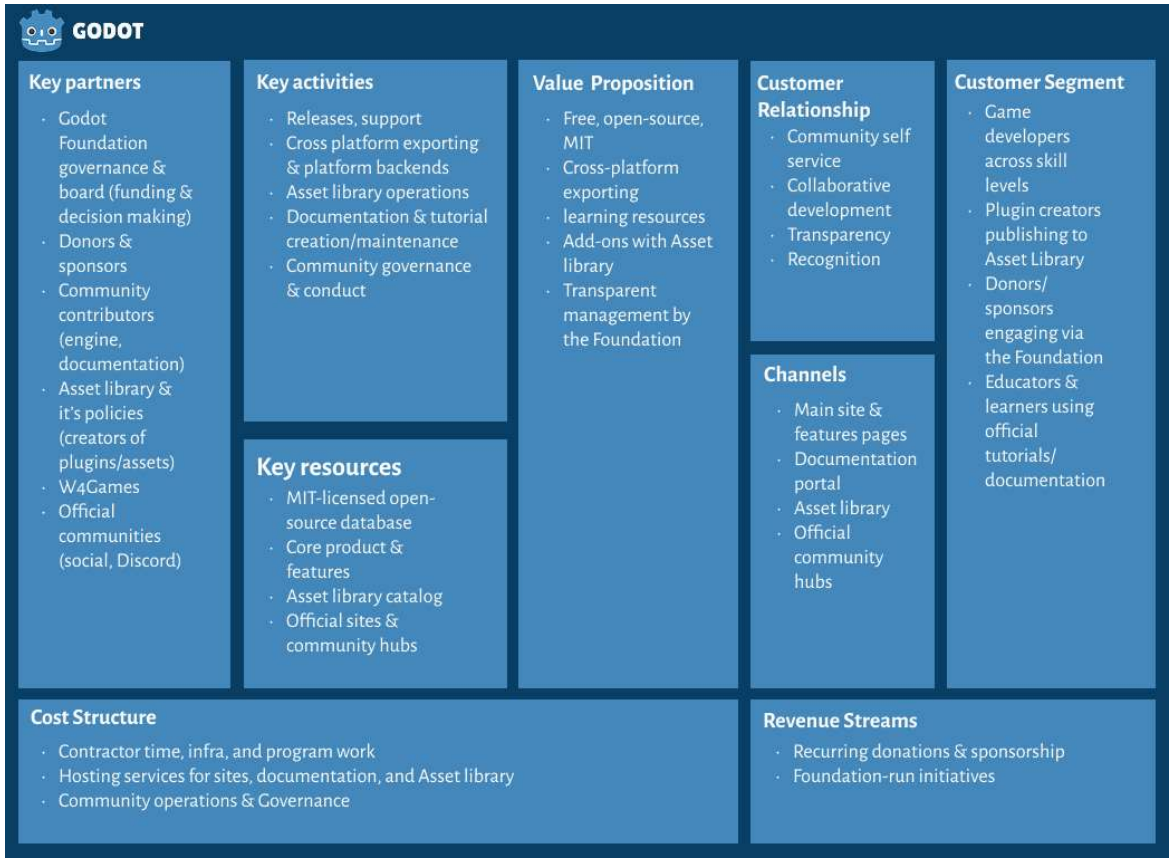



*Figure 8 Business model canvas summarising the key elements of the Godot ecosystem.*

### 1.4.1 Customer Segments & Value Proposition

Customer segments represent the groups an organisation chooses to serve (Osterwalder & Pigneur, 2010). Godot primarily targets game developers of all skill levels, from hobbyists to professional studios. Additionally, plugin and asset developers, donors, sponsors, educators, and students form key segments within the ecosystem. These groups both contribute to and benefit from Godot's open platform, fostering a strong community around the product (Godot Engine, n.d., Contributing-Godot Engine, n.d.)

The value proposition explains how a company creates value for its customers by solving problems or fulfilling needs (Osterwalder & Pigneur, 2010). Godot's core value lies in offering a free and open-source game engine under the MIT licence, providing full control, flexibility, and no royalties. This is complemented by cross-platform functionality, learning resources, an Asset Library with extensions, and transparent governance through the Godot Foundation. Together, these elements create economic and creative value for both developers and contributors (Godot Engine, n.d., Contributing – Godot Engine, n.d.)

### 1.4.2 Channels & Customer Relationships

Channels describe how a company communicates and delivers its value proposition to customers (Osterwalder & Pigneur, 2010). Godot primarily uses digital channels, including the official website, documentation portal, Asset Library, GitHub, and community platforms such as Discord. These channels support distribution, knowledge sharing, and community interaction with developers (Godot Engine).

Customer relationships define the type of connections a company establishes with its segments (Osterwalder & Pigneur, 2010). Godot's relationships are characterised by self-service through documentation, community support, and co-creation via open-source collaboration. Additionally, transparency in decision-making and recognition of contributors through changelogs and public communication strengthen community loyalty (Godot Engine, n.d., Contributing-Godot Engine, n.d.; Docs-Godot Engine, n.d.-h).

### 1.4.3 Revenue Streams & Key Resources

Revenue streams indicate how a company generates financial income (Osterwalder & Pigneur, 2010). As Godot is a non-profit organisation, its revenue primarily comes from donations and sponsorships through the Godot Foundation. These funds are used to finance development, infrastructure, and contract work. Potential future revenue sources could include workshops, events, and merchandising (Contributing - Godot Engine, n.d.-d; Godot Foundation, n. d. d.).

Key resources are the assets required to create and deliver the value proposition (Osterwalder & Pigneur, 2010). Godot's most important resources include the open-source codebase, core team of developers and contributors, Asset Library catalogue, and documentation and community platforms. These resources enable continuous innovation and the maintenance of a global ecosystem.

### 1.4.4 Key Activities & Key Partnerships

Key activities represent the actions a company must perform to make the model work (Osterwalder & Pigneur, 2010). For Godot, these include developing and maintaining the engine and platform, operating the Asset Library and ensuring quality, producing documentation and educational materials, governance, and community management. These activities support both technical quality and community engagement (Contributing - Godot Engine, n.d.; W4Games)

Key partners are the external actors who contribute to making the business model function according to Osterwalder & Pigneur (2010). For Godot, these include the Godot Foundation, W4Games, donors and sponsors, community contributors, plugin and asset developers, and official communities. Partners such as W4Games are essential partners, as they are a commercial company founded by core Godot contributors, which provides middleware console ports and enterprise support to studios using Godot, thereby extending the engines reach to proprietary console platforms while keeping the core project non-profit and open source (W4Games; Godot Foundation, n. d.).

### 1.4.5 Cost Structure

The cost structure encompasses the major expenses involved in operating the model (Osterwalder & Pigneur, 2010). Godot's primary costs relate to contract work, infrastructure (hosting, build services, documentation), and community management and governance. The model is value-driven rather than profit-oriented, with a focus on quality, accessibility, and community value.

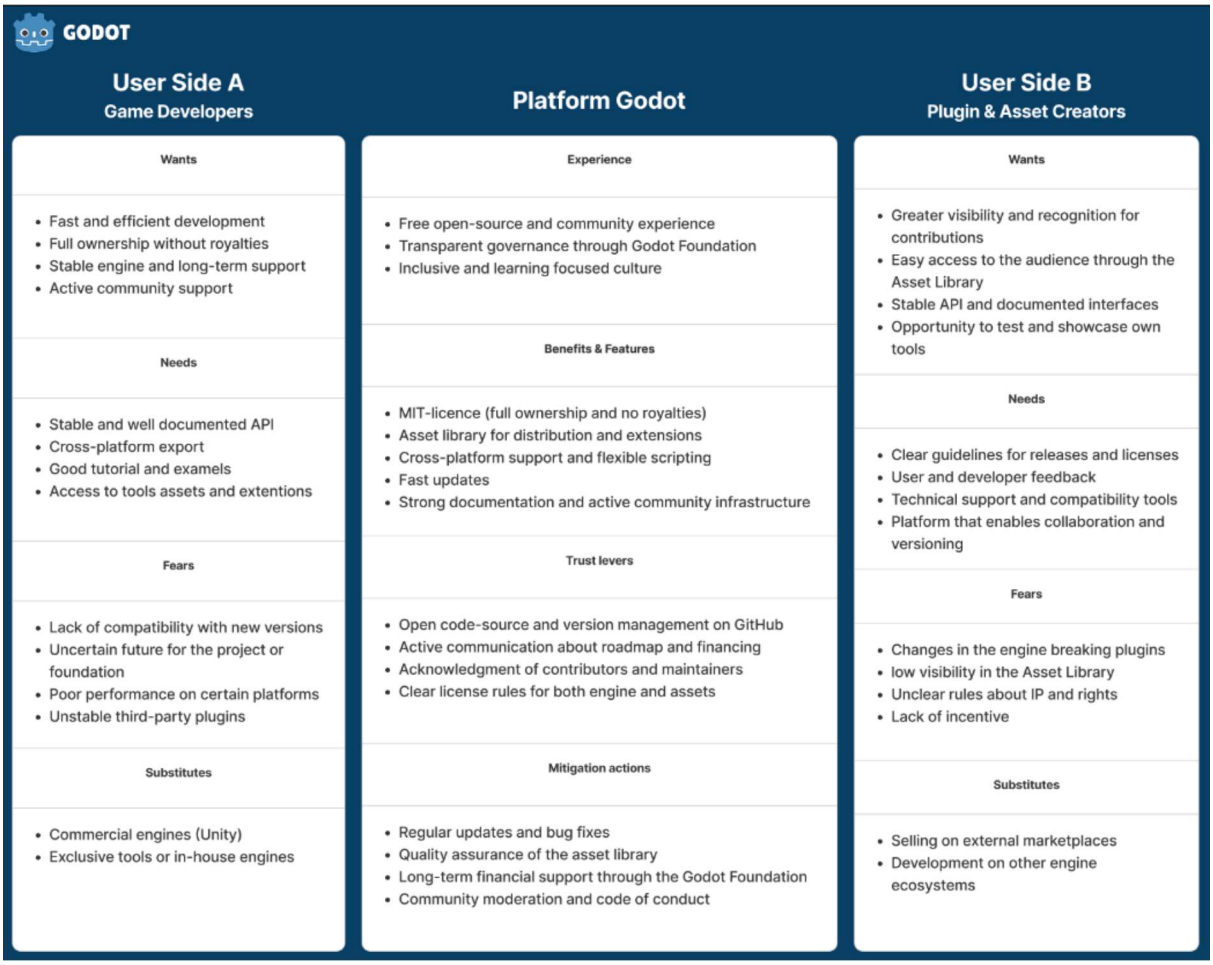



*Figure 9 Multisided Value Proposition Canvas for Godot.*

### 1.4.6 Multisided Value Proposition Canvas

A Multisided Value Proposition (MVPC) allows us to further into how value creation is balanced between the two main user sides and the platform itself (Belleflamme & Neysen, 2021). For *Users Side A*, which are the game developers, the MVPC shows that they **want** fast and efficient development, full ownership without royalties, and a stable engine with active community support. They **need** a well-documented API, cross platform export, good tutorials, and access to tools and assets. On the other hand, they **fear** version breakage, unstable third-part plugins and uncertain long-term support, making commercial engines such as unity or in-house tools relevant **substitutes**. In this case, W4Games helps address this sides fears, by offering a paid, professional solutions, which strengthens trust I Godot for commercial use.

*User Side B*, which are the Plugin and Asset creators, **wants** visibility and, recognition and easy publishing through the Asset Library. They **need** stable extension points, clear licensing guidelines, feedback and compatibility tools, but **fear** API changes, low visibility and unclear IP rules, with external marketplaces and other engine ecosystems as **substitutes**.

*The Godot platform* seeks to moderate these fears through a free, open-source experience, an MIT licence, the Asset Library, strong documentation and trust levers such as open code, a

public roadmap, and moderate community spaces. Thus, the MVPC shows that Godot must interlock these value propositions and mange cross-side tensions to sustain positive network effects between developers and creators (Belleflamme & Neysen, 2021).

The Business Model Canvas and Multisided Value Proposition Canvas are helpful visual tools for brainstorming. However, they offer a simplified view of the complex and changing interactions within platforms and are based on subjective opinions.

This business model analysis reveals how Godot balances value creation through community participation. How users experience this participatory model requires examining how the engine facilitates end-user development. The following section analyses how Godot supports the evolution from conventional tool use to collaborative development.

## 1.5 UX Evolution in Godot's Software Ecosystem: From Tool to Co-Design Platform

The Godot game engine shows how user experience evolves when software development The Godot game engine shows how user experience evolves as software development moves from traditional models to participatory ecosystems. Using Ardito et al.'s (2012) framework and Lieberman et al.'s (2006) end-user development paradigm, Godot has transformed from a conventional development tool into an environment where non-professional developers can create and extend game projects and tools.

### 1.5.1 The Shift Toward Meta-Design in Game Development

Godot's architecture follows meta-design principles. Instead of a linear *design-before-use* process, it supports what Lieberman et al. (2006) call 'evolutionary application development', where design continues during use within a *design-develop-use-evolve cycle*, see Figure 10 (Ardito et al. 2012).

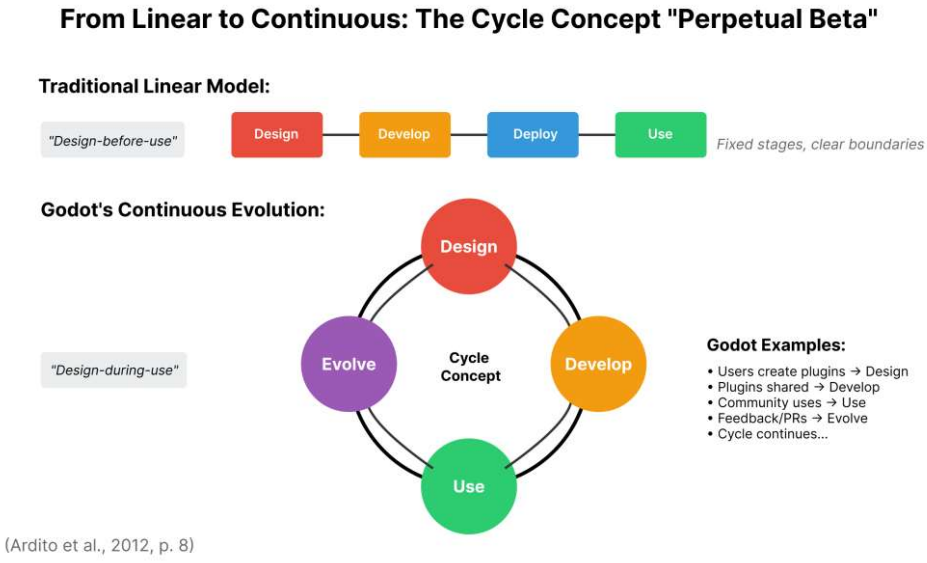



*Figure 10 Godot's Continuous Evolotion*

Users can configure game settings and rendering through visual menus for project settings and export presets (Doc-Godot Engine, n.d.-i). This matches Lieberman et al.'s (2006) first type of end-user activity, which is about selecting among predefined options without programming. The node system and GDScript work as application templates (Ardito et al. 2012). Users assemble reusable components into scenes, turn them into custom nodes, and share them through the Asset Library.

GDScript follow the *"gentle slope"* as shown in Figure 11, principle central to both papers. The language has Python-like syntax that reduces the learning curve while staying flexible enough for complex games. Users can begin with simple scripts on nodes, then gradually adopt signals, custom resources and finally GDExtension when needed. Each step increases power and complexity gradually, instead of forcing a jump to an entirely new language or paradigm (Lieberman et al., 2006).

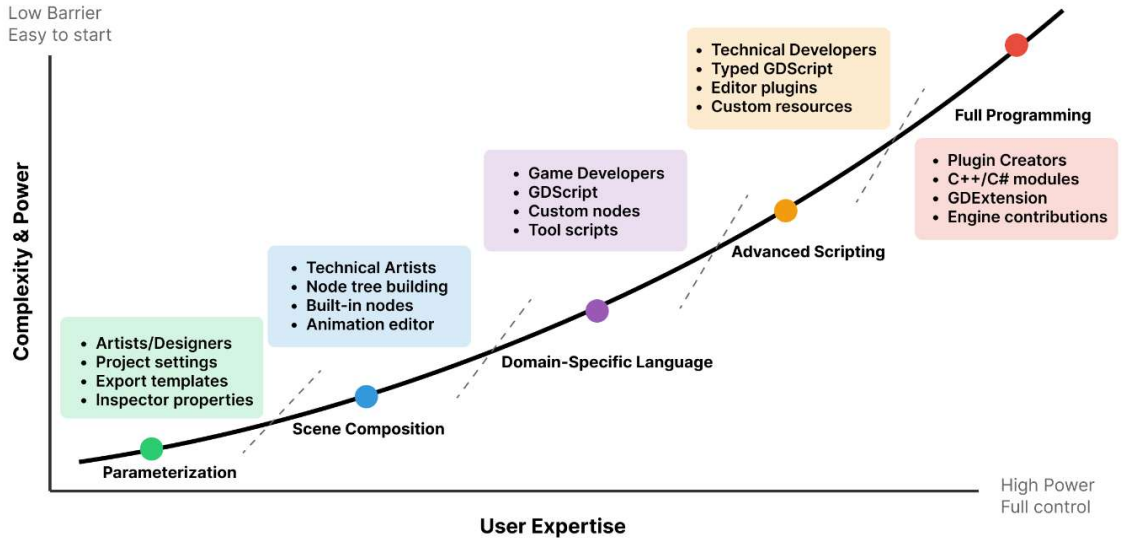



*Figure 11 The Godot "Gentle Slope" of EUD Complexity – Godot's end-user development progression from simple settings to code. (Lieberman et al, 2006, p 4)*

### 1.5.2 Addressing User Diversity Through Multiple Interaction Paradigms

User diversity challenges end-user development systems. Lieberman et al. (2006, p. 1) note that diversity includes different skills, knowledge, cultural backgrounds, and abilities, while Ardito et al. (2012, p. 5) note that even users of the same system often form different communities with distinct needs.

Godot handles this through multiple language options. GDScript is designed for game development with syntax built for common patterns like vector mathematics, scene management, physics, and event handling. C# support serves developers with existing expertise or performance requirements. C++ access through GDExtension enables low-level optimisation (Doc-Godot Engine, n.d.-j).

This multi-language approach matches Lieberman et al.'s (2006, p. 6) recommendation for domain-specific languages that use terms users already know. The engine's node-based architecture supports what Lieberman et al. (2006, p. 4-5) describe as successive decomposition: complex systems built from smaller, meaningful components. A character controller has movement, collision, animation, and sound nodes that each make sense

independently. Users can modify individual components without understanding the entire system and combine them in new ways (Godot Engine, n.d., Contributing – Godot Engine, n.d.).

### 1.5.3 The Ecosystem as Collaborative Infrastructure

Godot's asset library and plugin system creates collaborative infrastructure for end-user development. The Asset Library works as a repository where users share artifacts, from simple scripts to complete game templates (Godot Engine, n.d.-k). This supports the community-building that Lieberman et al. (2006, p. 5) call essential: spaces where end-users share knowledge and artifacts with peers.

Version control integration builds on these collaborative features. Godot's text-based file formats for scenes and project settings are designed to work with Git and similar tools, supporting the many-to-many distribution that Lieberman et al. (2006, p. 2) predicted would emerge. Users can copy projects, contribute improvements, merge changes, and fork repositories using standard development workflows.

The ecosystem moves from a model where a small number of professionals create tools for passive consumers to one where every user can potentially contribute. Ardito et al. (2012, p. 9) found that communication channels between different stakeholder groups help collaborative development and evolution. In Godot's case, these channels include GitHub discussions, community forums, Discord servers, and the proposals repository where users can suggest engine changes.

### 1.5.4 Implications for Software Product Development

The Godot ecosystem follows patterns predicted by theory. The community includes a great number of contributors who are not professional engine developers but create tools and improvements. This shows the exponential growth of end-user developers that Lieberman et al. (2006, p. 2) predicted.

Game developers routinely extend the editor through plugins, creating custom tools and dock panels (Godot Engine, n.d.-l). The boundary between using Godot and developing Godot disappears as users shape the tool to match their workflows.

End-user development in Godot has cultural and social dimensions. The international community has produced documentation translations and culturally diverse game projects. Practices evolve differently across communities while sharing common technical infrastructure.

The open-source model under the MIT licence gives developers freedom to study the engine’s code, and adapt it to their needs, not just share their own projects. This breaks down what Lieberman et al. (2006, pp. 2-3) call the traditional division between producers and consumers.

### 1.5.5 Gaps in the Current End-User Development Approach

While Godot's ecosystem shows many characteristics of successful end-user development, some gaps remain compared to theoretical ideals.

Users can start by adjusting settings and properties in the editor, then move to writing scripts. The engine provides both tools but learning how to make that jump depends on tutorials and community resources outside Godot. Community tutorials vary in quality and often show steps without explaining why. Users learn to follow instructions but struggle to adapt what they learned to different problems. The Asset Library lets people share plugins and assets but has no ratings or documentation standards. Users can download components but often can't figure out how they work.

Ardito et al. (2012, p. 9) argue software should adapt to users' skill levels and show only relevant tools. Godot's documentation doesn't do this. A beginner looking up "CharacterBody2D" sees the same technical reference as an expert would, with no indication if they should start elsewhere. The docs explain what features do but not how to learn them in order.

Sharing finished plugins and assets works well. However, learning how to create them is scattered across forums, Discord servers, YouTube channels, and community wikis. Users can find answers to specific questions, although understanding the ecosystem as a whole means piecing together information from many disconnected places.

Godot provides strong development tools. What's missing is learning support. With clearer learning paths and consolidated knowledge, more users could move from using Godot to actively contributing to it.

# Part 2: Designing a Learning Platform

## 2.1 Problem Statement

In the Part 1 we identified a gap in how Godot supports learning across different experience levels. The official documentation focuses on technical reference, while community knowledge is scattered across multiple platforms with limited structure. As a result, many users struggle to move beyond following isolated tutorials toward independent development.

Our design question is therefore: How might we design a learning platform that addresses these gaps and supports users' transition from tutorial-following to independent development?

To explore this, we use Godot as a case to investigate how ecosystem analysis can support the redesign of user-oriented support structures. In this section we examine what different user groups need from learning resources, what a learning platform requires to serve users across experience levels, and how such a platform can integrate with Godot's existing website and infrastructure. On this basis, we propose an integrated learning platform alongside the official documentation, with structured tutorial, curated learning paths, progress tracking and space for community driven tutorials that can be shared and rated in a more organized way, with the aim of lowering entry barriers and supporting the long-term sustainability of the Godot software ecosystem.

## 2.2 Theoretical Background

The learning platform builds on foundational concepts from Software Ecosystem (SECO) research, user-centred design, governance, and end-user development. A central challenge in SECOs is sustaining growth by lowering entry barriers and supporting continuous knowledge exchange. Dittrich (2014) highlights that sustaining an ecosystem requires supporting users beyond individual projects and ensuring access to shared norms, tools, and practices. In an open-source environment like Godot, a structured learning platform therefore becomes an important infrastructural element that reduces cognitive load and strengthens long-term sustainability.

Because ecosystems involve diverse stakeholder groups, the platform must create value across experience levels. Alves et al. (2017) show that community-based governance mechanisms - such as ratings, moderation, and review roles - help maintain contribution quality. The platform reflects this by offering accessible onboarding, structured progression, and channels for expert users to contribute tutorials.

User-generated tutorials connect the platform to End-User Development. Lieberman et al. (2006) show that non-professional developers play an important role in shaping digital infrastructures, and giving users the ability to create tutorials fosters engagement and shared ownership. To ensure the platform remains relevant and effective, continuous improvement is needed. Simonsen & Hertzum's (2022) effects-driven approach stresses the importance of measurable goals and iterative refinement. Metrics such as onboarding completion or tutorial engagement allow the platform to evolve in response to real-world use.

Finally, governance is essential in open-source environments. Markus (2007) highlights that transparent processes and distributed authority are key for coordinating contributions. The platform's built-in feedback loops and moderation structures reflect these principles, ensuring that community contributions remain aligned with the ecosystem's long-term direction.

Overall, the platform operates as a strategic ecosystem infrastructure: it lowers entry barriers, aligns stakeholder needs, enables community contributions, and embeds governance practices that support the ongoing growth and stability of the Godot ecosystem.

## 2.3 Stakeholders

The introduction of an integrated e-learning platform affects several key stakeholder groups within the Godot ecosystem. For the orchestrators, the Godot Core Team and Foundation, the platform reduces the burden of onboarding new users and strengthens long-term ecosystem sustainability by improving knowledge flow and presenting a more professional learning environment. Niche players, such as plugin and asset developers, benefit from users reaching technical literacy more quickly, which increases adoption of their tools and reduces support friction.

For external actors, the platform raises the overall competence of testers and bug reporters, leading to higher-quality feedback, while educational partners gain structured teaching material that facilitates institutional adoption. Vendors, including game studios and commercial partners, experience indirect benefits through a more capable talent pipeline and increased consistency in the quality of Godot-made games, contributing to the engine's credibility in professional settings. Finally, end-users, both beginner and professional developers, gain clearer onboarding, reduced frustration from *"tutorial hell,"* and better support for skill progression, ultimately improving the quality of games produced. See Figure 12 for the extensive Stakeholder Analysis.

| Category (M&H 2013) | Stakeholder Examples | Impact of E-Learning Platform on Stakeholder | What the Platform Needs from Stakeholder |
|---|---|---|---|
| **Orchestrator** | Godot Core Team; Godot Foundation | Reduces onboarding burden; strengthens sustainability by supporting structured knowledge flow; improves perception of professionalism and maturity. | Oversight for alignment with project vision; moderation guidance; integration with Docs/website infrastructure. |
| **Niche Players** | Plugin & tool developers; Asset library contributors | Users reach plugin/tool literacy faster; higher adoption of contributions; clearer version-aligned learning reduces support load. | Provide example workflows, optional tutorial contributions, feedback on advanced content needs. |
| **External Actors** | Testers; bug reporters; donors; educational partners | More skilled testers → better bug reports; institutions gain structured teaching material → higher educational adoption; donors see increased professionalism. | Share platform in educational settings; provide feedback on clarity and teachability; support adoption within institutions. |
| **Vendors** | Game studios using Godot; W4 Games; Asset store sellers | Strengthens talent pipeline; reduces training costs; improves consistency and quality of Godot-made games; | Communicate industry needs; validate advanced learning paths; showcase real-world use cases. |

| | | | |
|---|---|---|---|
| | | increases credibility for studios adopting Godot. | |
| **Customers (End-Users)** | Game developers using Godot; players of Godot-made games | Easier onboarding; more predictable progression; reduction of "tutorial hell"; improved game quality due to better-trained developers. | Engagement with tutorials; feedback on clarity; participation in rating/comment systems. |

*Figure 12 Stakeholder Analysis influence on E-Learning Platform (Manikas & Hansen, 2013, p. 1301)*

## 2.4 Design Research Methodology

We needed to understand what Godot users struggle with before designing solutions. Since no comparable learning platform exists within the Godot ecosystem, we built our understanding from research.

We first used AI-assisted desktop research to identify recurring learning and onboarding issues from the online communities, then validated the most common patterns through a survey. From there, we used Cooper et al.'s (2007) Goal-Directed Design framework to create personas and user journeys, which directly informed our Figma prototype.

This approach combines AI-supported analysis with human validation, a method validated by Li et al. (2024) and Madanchian & Taherdoost (2025) for UX research. It also reflects Lieberman et al.'s (2006) principles for End-User Development research and Simonsen & Hertzum's (2022) effects-driven approach, which means basing design decisions on actual user experiences rather than assumptions.

### 2.4.1 Desktop Research and Survey Validation

We conducted qualitative analysis of 63 community-generated texts from forums, blogs, learning platforms, and YouTube tutorial comments from 2019 to 2025, using AI to support the initial coding process. The AI tool sorted discussion themes, pain points, and learning challenges, which we then checked and refined manually to make sure the patterns were accurate and relevant.

Cooper et al. (2007, Ch. 4, p. 50) argue that surveys work best when they confirm patterns from qualitative research rather than standing alone. We followed this approach: after identifying patterns in community discussions, we created a survey to check whether these patterns held across different users. The survey mixed rating scales with open-ended questions, so we could both measure agreement and understand individual experiences. We posted it on the Godot Engine Discord server after getting moderator approval, and 24 community members responded (see appendix 2, attached PDFs).

The analysis identified three distinct user segments based on experience level, frustration patterns, and learning goals. Desktop research observations suggested beginners comprised approximately 40% of community discussions, intermediate users 35%, and experienced developers 25%. Our subsequent survey validation (n=24) showed a different distribution: beginners 33% (n=8), intermediate users 29% (n=7), and experienced developers 38% (n=9), suggesting experienced developers were more motivated to participate in structured feedback.

According to the survey, beginners struggled most with tutorials that show *"how"* without explaining *"why,"* making it hard to adapt what they learned to new situations. One respondent noted that *"tutorials don't care about good programming practices,"* while others struggled to *"search the docs for what I need without putting 'how to do X in Godot' into a*

*search engine."* Version confusion between Godot 3 and 4 came up repeatedly, with outdated tutorials causing code to fail.

Intermediate users showed what we termed *"tutorial hell syndrome"*, they had completed multiple tutorials but froze when facing blank projects. Survey respondents described a content gap where materials were *"either too basic or too advanced."* Documentation felt *"extremely vague"* with missing examples, especially for shaders and vector math. Several users discovered built-in Godot features months after coding their own versions, showing how hard it was to find what the engine already offered.

Experienced users wanted concept-focused guides on Godot's architecture, not beginner GUI tutorials. They requested C# and GDExtension documentation, low-level engine guides, and production-ready patterns. One developer said shader resources were *"quite poor". "Either you get a shrug with 'learn maths' or extremely basic tutorials."* Others pointed to years-old bugs missing from documentation and unclear differences between GDScript and C# implementations.

Version confusion affected everyone. Godot 4's breaking changes made it unclear which tutorials still worked. As one respondent put it: *"Godot is constantly updating, which means tutorials are constantly becoming outdated."* We saw this pattern in Part 1 (Figure 5 - The Upgrade Problem), where changes in the core engine affected the ecosystem, not just plugins but also the tutorials people rely on to learn, see appendix 2 for the references in this section.

### 2.4.2 Persona Development

The research revealed three distinct user groups. We turned these findings into personas using Cooper et al.'s (2007, Ch. 5) Goal-Directed Design framework. Cooper's method usually starts with ethnographic interviews, but we worked with our community discussion analysis and survey data instead.

We created three personas: beginner, intermediate, and experienced Godot users. Each persona is based on patterns we found in the survey responses.

We adapted Cooper's framework through five steps: (1) grouped users by experience level and behaviour patterns from community discussions, (2) checked these patterns through survey questions, (3) identified common frustrations, goals, and learning preferences in each group, (4) combined demographics, motivations, and pain points into complete personas, and (5) defined three goal types for each persona as Cooper (2007, Ch. 5, p. 92-94) describes: experience goals (how users want to feel), end goals (what they want to accomplish), and life goals (their broader aspirations in game development).

Tutorial Tina (See Figure 13) represents beginner users seeking to understand fundamental concepts without feeling overwhelmed. Her primary frustration is tutorials demonstrating *"how"* without explaining *"why,"* so she turns to resources like GDQuest to understand the concepts behind what she's doing.

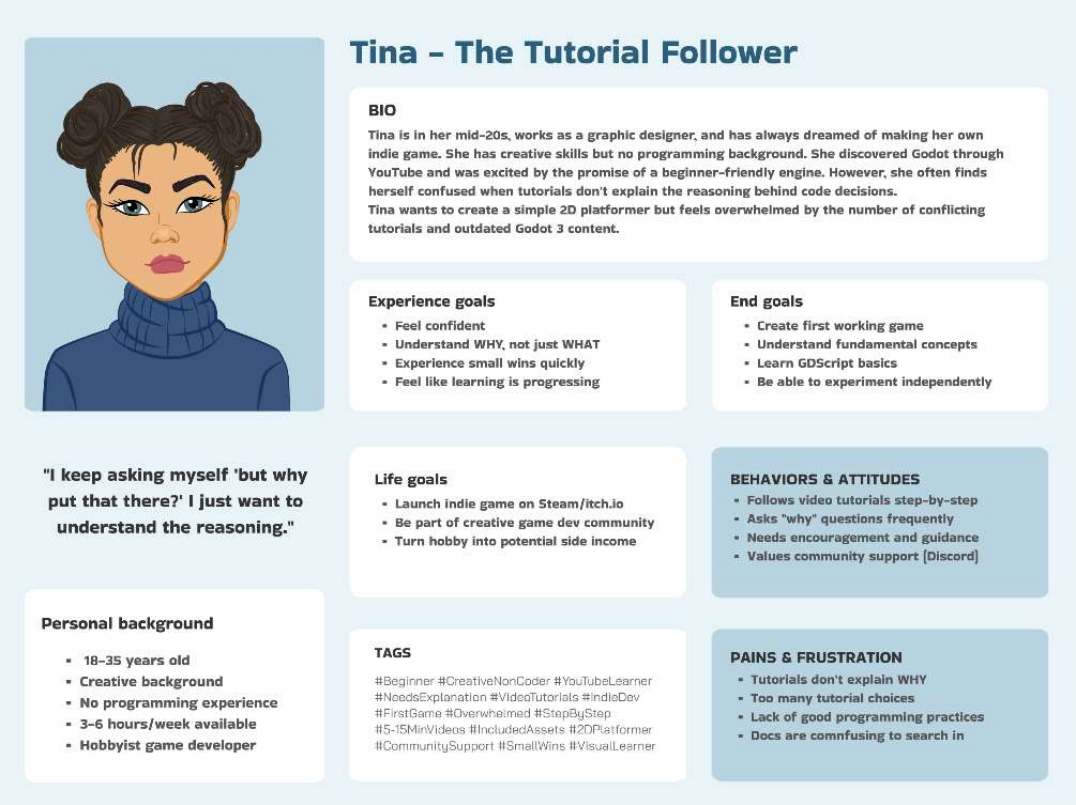



*Figure 13 Tina – The beginner persona*

Stuck Sam (intermediate) and Experienced Emma (experienced) address the other user segments identified in our research. Their detailed personas and needs are in appendix 3.

We created all three personas but focused the prototype on Tutorial Tina. She got a detailed user journey and fully interactive features. Sam and Emma's needs shaped our decisions about architecture and content structure, but we showed their experiences through mock-ups rather than full prototypes.

We chose Tina as our primary focus, which follows Cooper's principle (2007, Ch. 5, p. 104-106) of designing for one main persona while making sure the solution still works for others. This choice made sense for two reasons: we had limited resources, and solving beginner problems like missing explanations helps everyone. When beginners understand the *"why"* behind concepts, intermediate and experienced users benefit from that clearer foundation too.

### 2.4.3 User Story Mapping

User story mapping is a UX tool for visualising how a user moves form an initial situation towards a goal through a sequence of activities, decisions, and touchpoints. In our case we used users story mapping (see Figure 14), where the story is structured from left to right as goals, actions and tasks, to maintain both overview and detail in a single artifact (Patton & Economy, 2014). Besides Figure 14, we also created a User Story Map for the other two personas as seen in appendix 4.

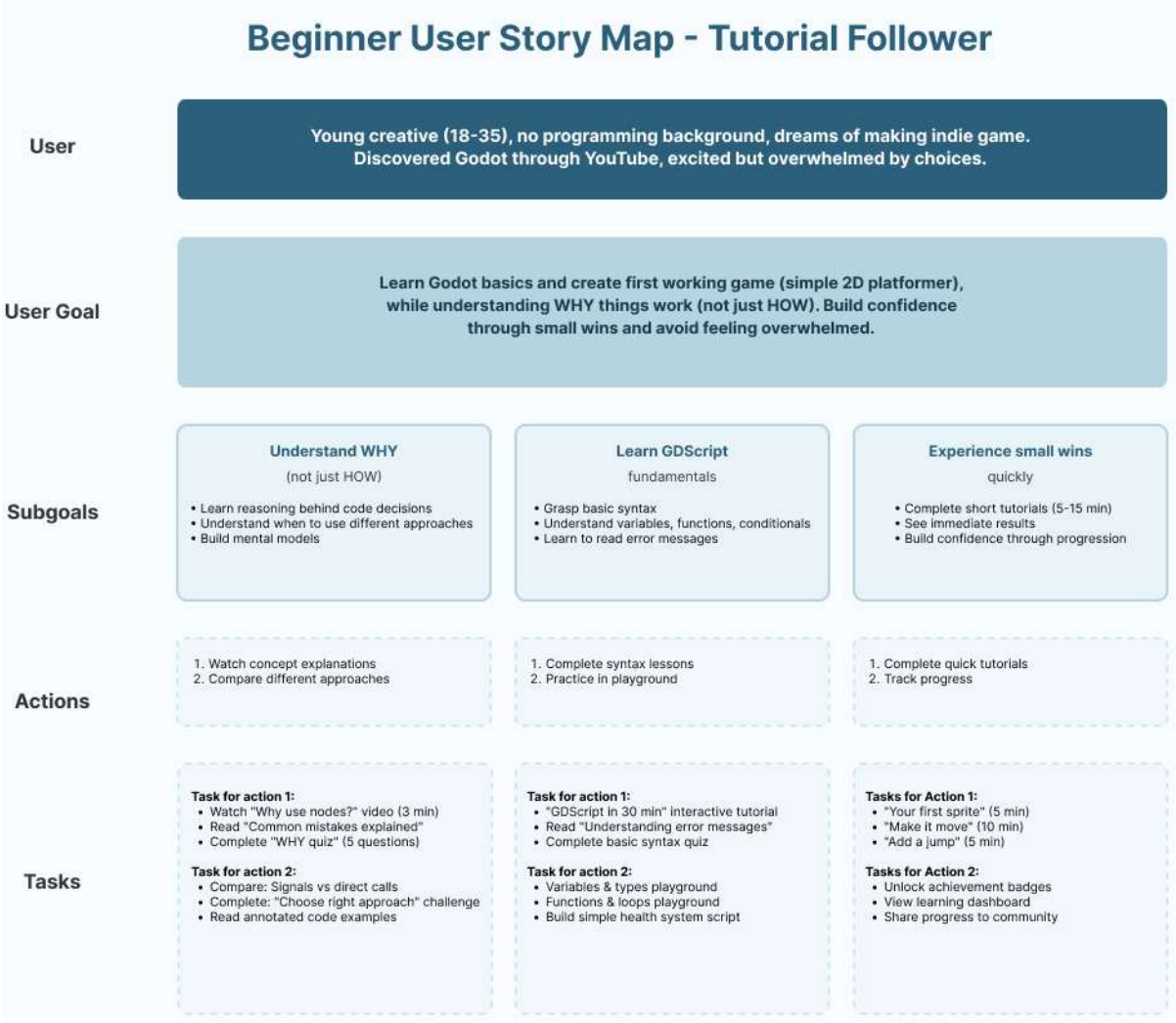



*Figure 14 Tina – The beginner User Story Map*

For each of the personas, beginner, intermediate and experienced user, we first defined a short persona description and overall goal at the top row of our visual user journey map. We then identified 3 sub-goals that describe major phases in their learning journey, such as *"Understand WHY"*, *"Start projects from scratch"*, or *"Leverage advanced features"*. Under each sub-goal we listed high-level actions the user would take and finally broke these down into concrete tasks that the designed learning experience should offer. This preserved both the big picture and the detailed steps, which is a core benefit of story mapping (Patton & Economy, 2014).

The maps highlight different pain points and motivations at each expertise level: beginners need confidence and quick wins, intermediates need help escaping tutorial dependency, and experienced users need concept-driven, code-first resources. These insights directly informed design decisions such as structuring the learning path into level-specific tracks, prioritising concept guides and challenges for non-beginners, and designing features like progress tracking and *"from scratch"* templates to support key moments in each (Patton & Economy, 2014).

Based on these research insights and stakeholder perspectives, we now turn to the business model for the proposed solution. In the following, we use the identified user needs, pain

points, and ecosystem roles as a basis for describing how the solution is intended to create, deliver, and capture value within the Godot software ecosystem.

## 2.5 Business model

Our design solution creates value by curating scattered Godot learning resources into clear beginner, intermediate, and expert paths, helping users quickly find tutorials that match their level and reducing the potential challenges of getting started. We deliver this value through a browser-based LMS that combines text, exercises, and embedded video, and links directly to official Godot documentation. A key element of the model is collaboration with YouTubers who are well-known in the Godot community: their videos are embedded into our tutorials, and joint promotions increase visibility for both the creators and the platform. Additional exposure is generated through PR campaigns and collaborations with other recognised figures in the Godot ecosystem.

## 2.6 Current Godot Website Architecture and Proposed Addition

The official online presence for Godot is split between two main sites: the frontpage at godotengine.org and the documentation at docs.godotengine.org. Both are implemented as static sites. The frontpage is generated with build tool Jekyll, while the documentation uses Sphinx and reStructuredText. Instead of a traditional database-driven CMS, Godot uses Git repositories plus static site generators as a file-based CMS, where content is written as text files in Git, and the build tools generate the final HTML (GitHub. 17.12.2025.-a). The architecture of the website can be seen in Figure 15.

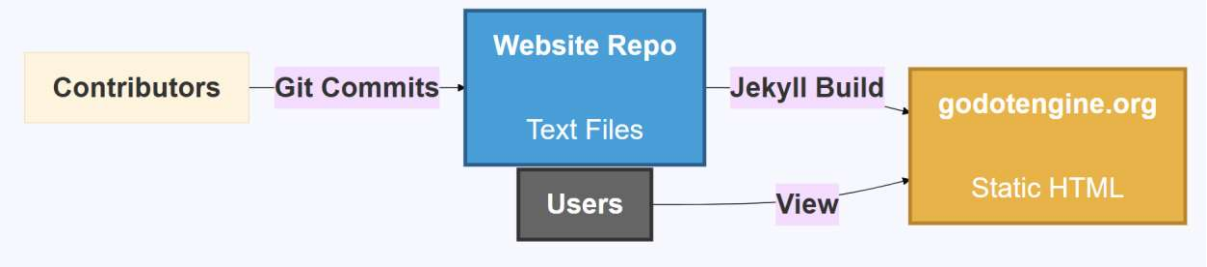



*Figure 15 Diagram showing the current architecture of the Godot website frontpage*

The documentation repository is organised into directories such as tutorials/, classes/, and getting_started/. Contributors would work directly in reStructuredText files, and Sphinx and builds the published documentation. A set of Sphinx extensions adds practical features such as code examples, and custom support for GDScript-specific syntax (GitHub. 17.12.2025.-a) The architecture of the documentation can be seen in Figure 16.




*Figure 16 Diagram showing the current architecture of the Godot documentation*

For comments and feedback, the documentation embeds Giscus as a client-side widget instead of running a custom backend. The docs themselves remain static, but Giscus connects to a separate GitHub repository that stores user notes as GitHub Discussions in GitHub's own database. The widget loads in the user's browser, links each documentation page to the corresponding discussion and displays the notes at the bottom of the page. This keeps user-contributed notes associated with the documentation without introducing a database or comment system in the infrastructure (GitHub. 17.12.2025.-a).

Search is handled by Sphinx's built-in search system. A search index is generated at build time, and the search box in the sidebar queries this index. The current solution is functional but basic, as there is no autocomplete or search-as-you-type, and results only appear after submitting a query (GitHub. 17.12.2025.-a).

Our project proposal builds on this structure by adding a dedicated tutorial page that connects to the main website and a simple learning management system (LMS). Users would be able to create an account so the site can remember which tutorials they have completed. Authentication and progress storage can be handled by an external service and a small backend with a database to store user progress. Tutorials would embed playable videos hosted on YouTube via the YouTube IFrame Player API. To make the learning experience more interactive, we also propose embedding the Godot Web Editor directly into the tutorial page so that users can write GDScript and run scenes in the browser alongside the explanations (Godot Web Editor. 02.12.2025).

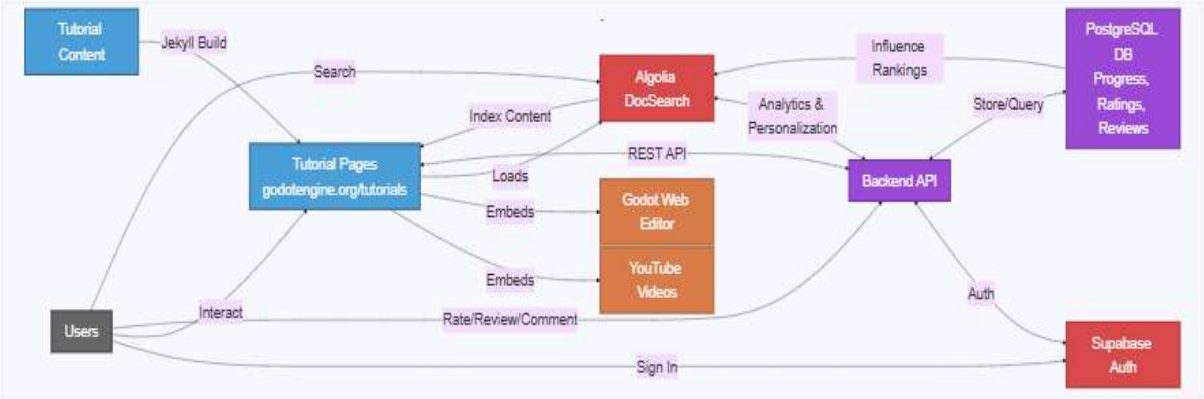



*Figure 17 A diagram showing the architecture of the proposed additions*

To improve search, we propose replacing the current Sphinx search with a smart search solution such as Algolia DocSearch. Algolia offers hosted, instant search with autocomplete, relevance ranking, and typo tolerance, and it can index static documentation sites without requiring custom backend infrastructure (Algolia, n.d.).

Integrating Algolia DocSearch would mainly involve adding its JavaScript library to the existing site, configuring it to search the docs.godotengine.org with appropriate filters, and connecting it to the current search field. The autocomplete dropdown could show highlighted matches grouped by different tags. Over time, search analytics and the LMS could be combined to personalise search results, for example by highlighting tutorials that match a user's level or learning path.

On top of this, we propose a lightweight rating and comment system for community-created tutorials. Each tutorial page would include a small widget that lets logged-in users assign a rating and optionally leave a short text review. The LMS backend and existing user accounts would be reused to store these ratings and comments in the same database as progress data, so that each tutorial can display an aggregated score, number of reviews and recent comments. Ratings can also feed back into the search and recommendation logic, so that highly rated tutorials are shown more in search results. The architecture is shown in Figure 17.

## 2.7 Prototype and Design Choices

We used a step-by-step approach for our design. First, we sketched key desktop screens on a whiteboard as low fidelity prototypes (see Figure 18). These simple drawings let us explore ideas quickly and change layouts without spending time on visual details (Hartson & Pyla, 2018).

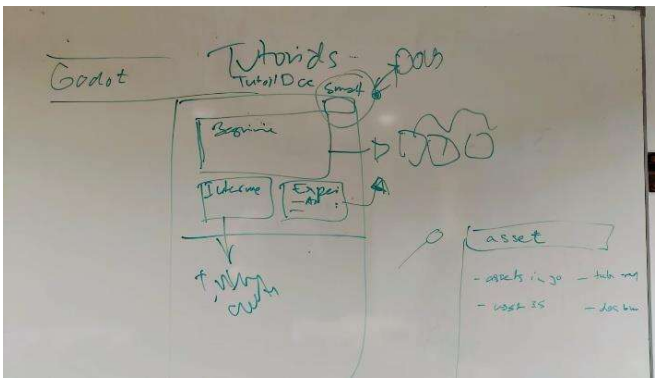



*Figure 18 Sketch of prototype*

Next, we recreated the main screens as interactive desktop wireframes in Figma, so we could click through the interface and test basic navigation and tasks. The prototype focuses on the beginner user journey, where the focus is to guide the user through their learning. Focusing on this flow is important because first-time use and early tasks are critical moments; if beginners struggle here, they may abandon the system before seeing its benefits.

The prototype is intended as a conceptual design artifact rather than a fully implemented system. Its purpose is to demonstrate how insights from user research and ecosystem analysis can be translated into concrete interface structures and feature concepts rather than to validate usability through deployment or testing.

### 2.7.1 Smart Search

A core design choice is the Smart Search feature, which integrates search results from both the official Godot documentation and the community-generated tutorials. This addresses a major finding from our user research: learners often struggle to understand *where* to search and *which* resources are appropriate for their current skill level. Smart Search helps by providing a unified search interface that clearly distinguishes between the search results for the text-based documentation and the video tutorial-based content. This helps the user to navigate the ecosystem more confidently.

Instead of separating documentation and tutorials into different search entry points, we chose a unified search solution to reduce cognitive load for beginners, who may not yet understand the boundaries between different knowledge sources in the ecosystem. Visual distinction between result types preserves clarity while keeping all relevant resources accessible in one place.

### 2.7.2 Rating and Comments

The rating and comments feature is another central design element to help users quickly identify high-quality tutorials and to reduce the uncertainty many learners expressed in our research. Users often struggle to judge whether a tutorial is accurate, up to date, or suitable for their experience level. By allowing viewers to rate tutorials and leave short comments, the platform creates a simple feedback loop that highlights helpful resources and signals when content may be outdated. This also supports a culture of shared learning, since users can point out version differences, missing steps, or useful tips. Over time, the rating system helps maintain overall quality and makes it easier for learners to choose tutorials they can trust.

### 2.7.3 Wireframe description

The following is a description of our prototype for each wireframe. The prototype is a low-level interactive desktop web experience built in Figma that adds a structured learning hub to the Godot website, with a focus on the beginner path. It does not aim to represent the final visual design or full functionality; instead, the intention is to demonstrate the overall flow of the learning experience from a beginner's perspective. Clickable elements are limited to key steps in the journey (e.g. choosing a path, starting the "Understand WHY" step, opening a lesson), so that we can focus on evaluating the structure and guidance the interface provides to new Godot users.

**Landing page - "Choose Your Learning Path":** From the main navigation, users select How to Godot and arrive at a dedicated landing page (wireframe-5). Here, three cards, beginner, intermediate, and expert present short descriptions and example topics, giving an immediate sense of *"where do I fit?"* This directly addresses survey feedback that new users feel overwhelmed by scattered tutorials and unsure where to start. A global search bar and *"What's New"* panels (docs, news, trending tutorials) support more experienced users who want quick access to specific content without losing the curated overview. Logged-in users can also see their progress and saved tutorials (wireframe-9/10), countering the pain point of not knowing what they have already covered.

**Beginner flow - "Understand WHY":** Clicking Start Learning on the Beginner card opens a three-step path: Understand WHY, Learn GDScript, and Experience Small Wins (wiredframe-6). This structure is based on the beginner persona and user journey, which highlighted a lack of conceptual understanding and reliance on copy-pasting code. In the Understand WHY step (wireframe-7), short actions and tasks, such as watching concept explanations and comparing approaches are presented as checkable cards, with a progress bar, duration estimate, and rating badge. This helps beginners build mental models while seeing tangible progress, reducing anxiety and *"tutorial hell"*.

**“Learn GDScript” module:** The second step focuses on core language skills. Lessons combine embedded YouTube videos, text explanations, and small coding exercises (wireframe-8). Progress tracking and ratings again help users choose trusted material and stay motivated, responding to survey comments about inconsistent tutorial quality.

**Interactive playground:** Within Learn GDScript and Experience Small Wins, an embedded Godot Web Editor acts as an interactive playground (wireframe-8). Users can type GDScript and run scenes directly inside the tutorial page, closing the gap between watching and doing. This directly targets the pain point that existing tutorials often remain theoretical and require complex local setup before learners can experiment.

Each tutorial displays a compact rating panel at the top showing an average star rating, number of reviews, estimated time, and number of learners who completed it. This helps users quickly judge quality and effort before committing. Below the content, a Feedback & Questions section allows learners to post comments, ask questions, or report issues. Community members and maintainers can reply, upvote helpful answers, and mark solutions, turning each tutorial into a small discussion space that continuously improves the material.

The prototype was not user tested. Validation and iteration based on real user interaction are therefore identified as important next steps and are discussed further in the Reflection section.

Figma link to the interactive prototype:
https://www.figma.com/proto/n125zsp8DTjISVXb8KbL9T/SECO?node-id=452-957&p=f&t=vH9uCeKyhfrHgW6i-1&scaling=min-zoom&content-scaling=fixed&page-id=452%3A941

## Reflection

The analysis of the Godot software ecosystem revealed that, while the engine is growing and technically maturing, the learning resources are fragmented and not ideally aligned with users' experience levels. Official documentation mostly serves as technical reference, while other knowledge is distributed across many different channels. This fragmentation makes it difficult, especially for beginners to start developing a solid understanding of the engine. Our design was motivated by the gap between the available knowledge and user's ability to utilize it. These findings, together with the architecture, UX and stakeholder analyses in Part 1, we defined a design question for Part 2: How might we design a learning platform that addresses these gaps and supports users' transition from tutorial-following to independent development?

Our design provides a concrete answer to this question. The proposed learning platform reorganises scattered resources into beginner, intermediate and expert paths and links them to personas and user stories derived from community research. Smart Search directly targets the fragmentation problem by unifying documentation into a single search system. The LMS layer adds progress tracking and structured modules, which address users' need for guidance beyond isolated tutorials. Rating and comment features address the lack of quality ratings for community content, while the ability to contribute to user driven tutorials as volunteer educator aligns with end-user development and governance perspective in the Part 1.

Our prototype has not been user tested, validation is based on user research and personas rather than observed behavior, we only did extensive prototyping on the beginner journey, and the technical feasibility and performance of Smart Search remain untested. Despite this, the design shows how ecosystem and stakeholder analysis can inform concrete design decisions rather than staying at an abstract level, and it highlights learning as a key form of participation in the Godot ecosystem. Future work should include task-based usability tests and think-aloud tests with real users, A/B tests on Smart Search and rating layouts, and collection of usage data on comments and ratings. We also see potential in adding personalization features, supporting community contributions to the learning platform, improving accessibility, and involving core contributors, plugin authors and educators in co-design workshops.

# References

Alves, C., Oliveira, J., & Jansen, S. (2017). *Software Ecosystems Governance*.

Ardito, C., Buono, P., Costabile, M. F., Lanzilotti, R., & Piccinno, A. (2012). End users as co-designers of their own tools and products. *Journal of Visual Languages & Computing*, *23*(2), 78–90.

Belleflamme, P., & Neysen, N. (2021). A multisided value proposition canvas for online platforms. *Journal of Business Ecosystems (JBE)*, *2*(1), 1–14.

Bryson, J. M. (2004). What to do when stakeholders matter: Stakeholder identification and analysis techniques. *Public Management Review*, *6*(1), 21–53.

Burton, R. (2025, July 11). *The Rise of Godot*. Game Oracle. https://www.game-oracle.com/blog/the-rise-of-godot

Cooper, A., Reimann, R., & Cronin, D. (2007). *About face 3: The essentials of interaction design*. John Wiley & Sons.

Dittrich, Y. (2014). Software engineering beyond the project–Sustaining software ecosystems. *Information and Software Technology*, *56*(11), 1436–1456.

Eden, C., & Ackermann, F. (1998). *Making strategy: The journey of strategic management*. Sage.

Hartson, R., & Pyla, P. S. (2018). *The UX book: Agile UX design for a quality user experience*. Morgan Kaufmann.

Jansen, S., Brinkkemper, S., & Finkelstein, A. (2013). Business network management as a survival strategy. In *Software Ecosystems* (pp. 29–42). Edward Elgar Publishing.

Li, J., Cao, H., Lin, L., Hou, Y., Zhu, R., & El Ali, A. (2024). User experience design professionals' perceptions of generative artificial intelligence. *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*, 1–18.

Lieberman, H., Paternò, F., Klann, M., & Wulf, V. (2006). End-user development: An emerging paradigm. In *End user development* (pp. 1–8). Springer.

Madanchian, M., & Taherdoost, H. (2025). The impact of artificial intelligence on research efficiency. *Results in Engineering*, 104743.

Manikas, K., & Hansen, K. M. (2013). Software ecosystems–A systematic literature review. *Journal of Systems and Software*, *86*(5), 1294–1306.

Markus, M. L. (2007). The governance of free/open source software projects: Monolithic, multidimensional, or configurational? *Journal of Management & Governance*, *11*(2), 151–163.

Osterwalder, A., & Pigneur, Y. (2010). *Business model generation: A handbook for visionaries, game changers, and challengers*. John Wiley & Sons.

Patton, J., & Economy, P. (2014). *User story mapping: Discover the whole story, build the right product.* O'Reilly Media, Inc.

Pruijt, L., & Brinkkemper, S. (2014). A metamodel for the support of semantically rich modular architectures in the context of static architecture compliance checking. In *Proceedings of the WICSA 2014 Companion Volume* (pp. 1–8).

Sestoft, P., & Vaucouleur, S. (2008). Technologies for evolvable software products: The conflict between customizations and evolution. In *Advances in Software Engineering: Lipari Summer School 2007, Lipari Island, Italy, July 8-21, 2007, Revised Tutorial Lectures* (pp. 216–253). Springer.

Simonsen, J., & Hertzum, M. (2022). Effects-Driven IT Improvement: Pursuing local post-implementation opportunities. *Scandinavian Journal of Information Systems*, *34*(1), 37–70.

Unphon, H., & Dittrich, Y. (2010). Software architecture awareness in long-term software product evolution. *Journal of Systems and Software*, *83*(11), 2211–2226.

**Links:**

Algolia, (n.d). Algolia Documentation. Retrieved December 17, 2025, from:

https://www.algolia.com/doc

Contributing - Godot Engine. (n.d.). How to contribute. Retrieved October 18, 2025, from: https://contributing.godotengine.org/en/latest/organization/how_to_contribute.html

Docs - Godot Engine. (n.d.-a). Overview of Godot's key concepts. Retrieved October 18, 2025, from: https://docs.godotengine.org/en/stable/getting_started/introduction/key_concepts_overview.html

Docs - Godot Engine. (n.d.-b). Nodes and scenes. Retrieved October 18, 2025, from: https://docs.godotengine.org/en/stable/getting_started/step_by_step/nodes_and_scenes.html

Docs - Godot Engine. (n.d.-c). Using signals. Retrieved October 18, 2025, from: https://docs.godotengine.org/en/stable/getting_started/step_by_step/signals.html

Docs - Godot Engine. (n.d.-d). Optimization using servers. Retrieved October 18, 2025, from: https://docs.godotengine.org/en/stable/tutorials/performance/using_servers.html

Docs - Godot Engine. (n.d.-e). What is GDExtension? Retrieved October 18, 2025, from: https://docs.godotengine.org/en/stable/tutorials/scripting/gdextension/what_is_gdextension.html

Docs - Godot Engine. (n.d.-f). Scene organization. Retrieved October 18, 2025, from: https://docs.godotengine.org/en/stable/tutorials/best_practices/scene_organization.html

Docs - Godot Engine. (n.d.-g). Upgrading from Godot 3 to Godot 4. Retrieved October 18, 2025, from: https://docs.godotengine.org/en/stable/tutorials/migrating/upgrading_to_godot_4.html

Docs - Godot Engine. (n.d.-h). Godot Docs – 4.5 branch. Retrieved October 18, 2025, from: https://docs.godotengine.org/en/stable/index.html

Docs - Godot Engine. (n.d.-i). Project settings. Retrieved November 12, 2025, from: https://docs.godotengine.org/en/stable/tutorials/editor/project_settings.html

Docs - Godot Engine. (n.d.-j). Cross-language scripting. Retrieved November 12, 2025, from: https://docs.godotengine.org/en/stable/tutorials/scripting/cross_language_scripting.html

Docs - Godot Engine. (n.d.-k). About the Asset Library. Retrieved November 12, 2025, from: https://docs.godotengine.org/en/stable/community/asset_library/what_is_assetlib.html

Doc - Godot Engine. (n.d.-l). Making plugins. Retrieved November 12, 2025, from: https://docs.godotengine.org/en/stable/tutorials/plugins/editor/making_plugins.html

Docs - Godot Engine (n.d.-x). Picture of Godot's architecture diagram. Retrieved December 17, 2025, from: https://docs.godotengine.org/en/4.4/contributing/development/core_and_modules/godot_architecture_diagram.html

GDScript. (2025) Signals. Retrieved December 3, 2025, from: https://gdscript.com/solutions/signals-godot/

GitHub. (17.12.2025.-a). Godot Website GitHub Repository. Retrieved December 17, 2025, from: https://github.com/godotengine/godot-website

GitHub. (17.12.2025.-b). Godot Documentation GitHub Repository. Retrieved December 17, 2025, from: https://github.com/godotengine/godot-docs/

Godot Engine. (n.d). Consoles. Retrieved October 18, 2025, from: https://godotengine.org/consoles/

Godot Foundation. (n.d.). Godot Foundation. Retrieved October 18, 2025, from: https://godot.foundation/

Godot Web Editor. (02.12.2025). Godot Web Editor. Retrieved December 17, 2025, from: https://editor.godotengine.org/releases/latest/

Governance – Godot Engine. (n.d.). Governance. Retrieved November 15, 2025, from: https://godotengine.org/governance/

W4Games. (n.d.). Why Godot? Retrieved October 18, 2025, from: https://www.w4games.com/why-godot

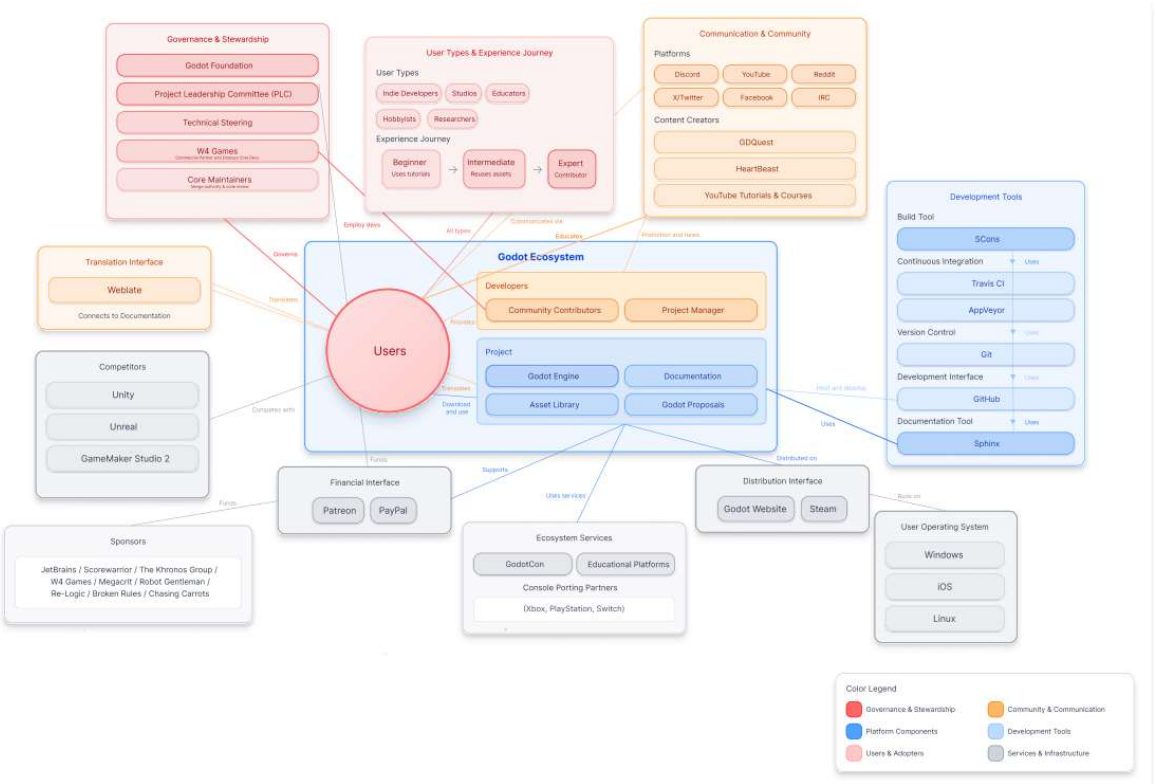
W4Games. (n.d.). W4Games. Retrieved October 18, 2025, from: https://www.w4games.com/

# Appendix

To explore and zoom in on the figures and models, go to the Figma page: https://www.figma.com/design/n125zsp8DTjISVXb8KbL9T/SECO?node-id=452-3&t=ZIxetgMAJVc2aaAr-1

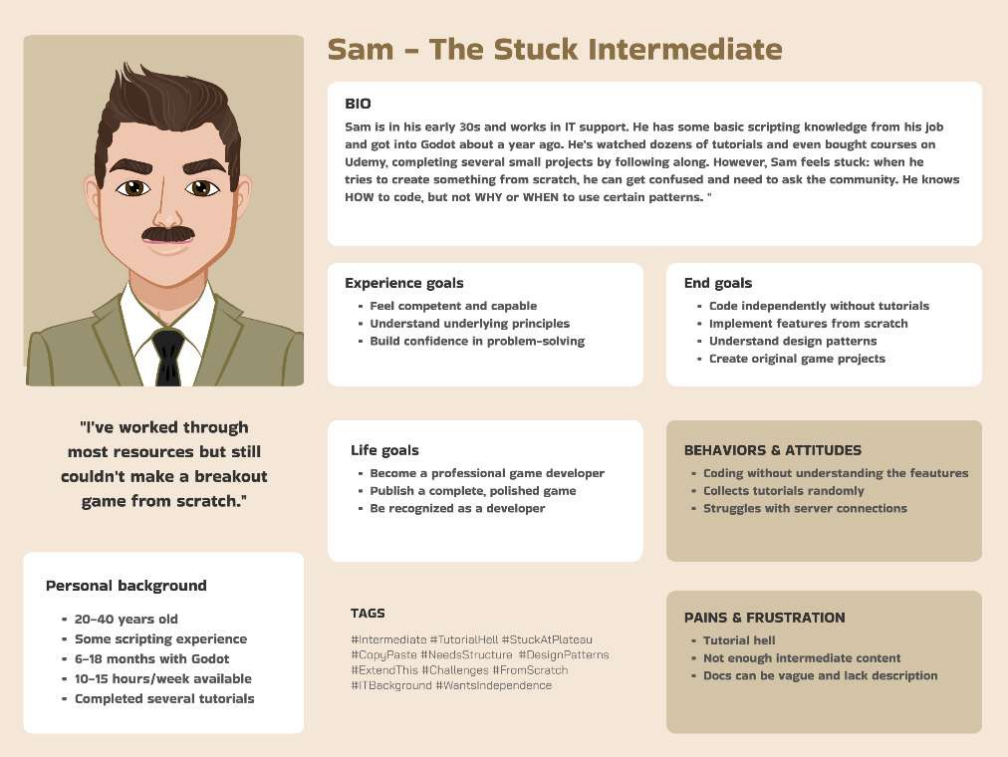
## 1. Ecosystem

The Godot Software Ecosystem. Users interact with the Godot engine, documentation, and Asset Library, governed by the Godot Foundation and supported by an active community across Discord, YouTube, and forums, with development coordinated through GitHub and funded via Patreon.

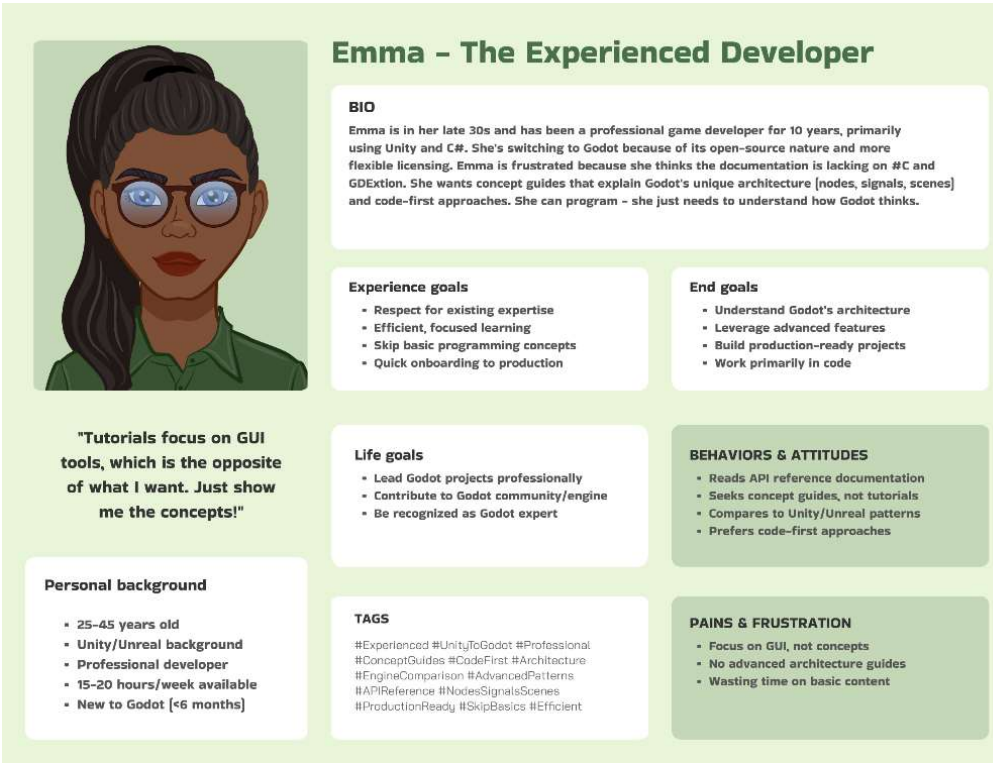



## 2. Personas

Sam - The intermediate persona

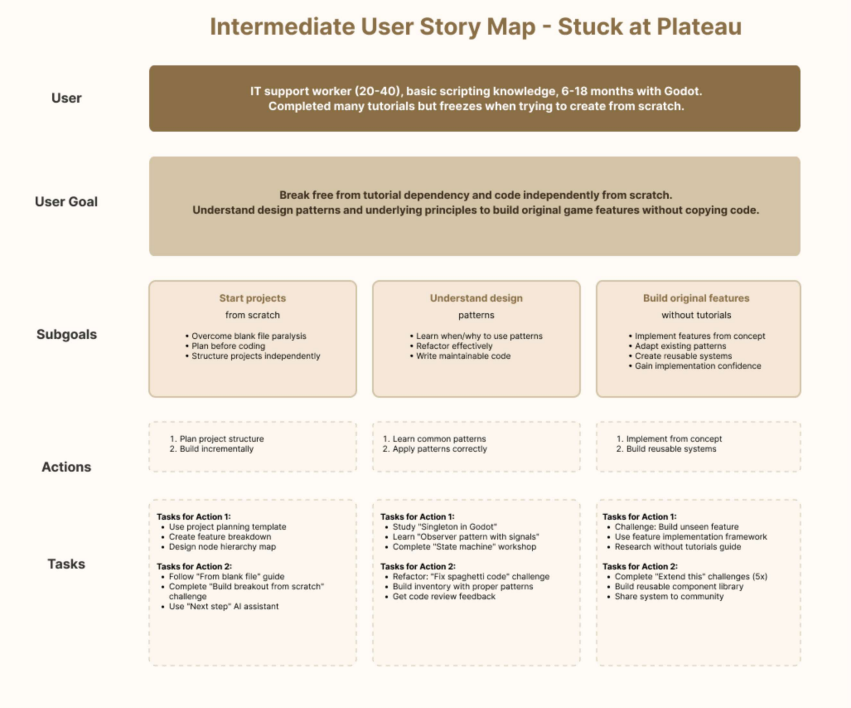



Emma - The experienced persona

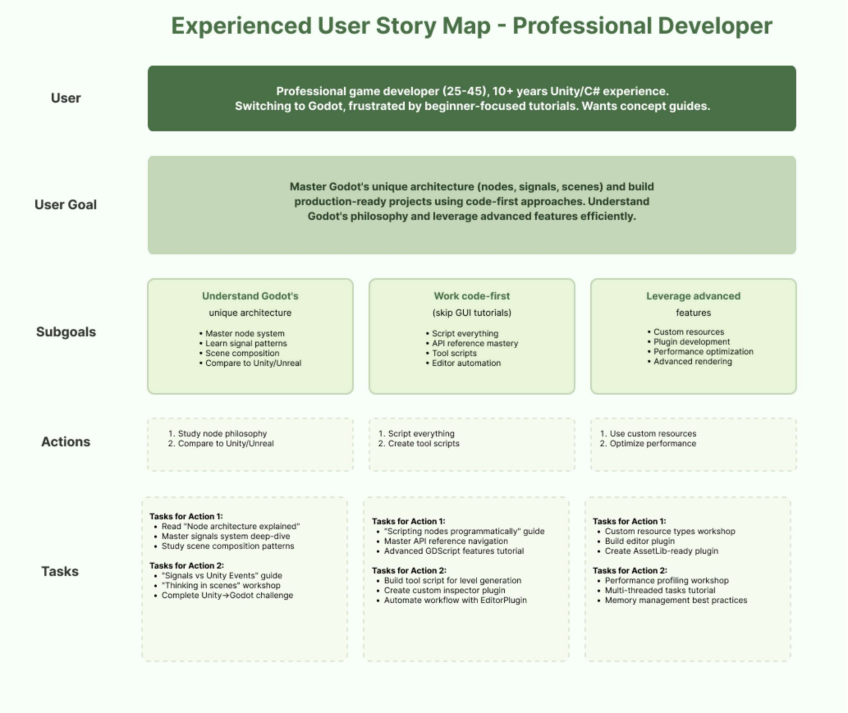



## 3. User Story

Sam's User Story:




Emma's User Story:




## 4. Surveys:

See the attached three PDFs for Beginner, Intermediate and Experienced users.